

Cluster analysis

Aliaksandr Hubin

Mar 10 2026

Chapters for cluster analysis

ISLR: Chapter 12: Unsupervised Learning

- o Section 12.4: Clustering Methods
 - § Section 12.4.1: K-Means Clustering
 - § Section 12.4.2: Hierarchical Clustering

R Book: Chapter 25: Multivariate Statistics

- o Section 25.3: Cluster Analysis
 - § Section 25.3.1: Partitioning
 - § Section 25.3.2: Taxonomic Use of K-means
- o Section 25.4: Hierarchical Cluster Analysis

Topics for this lecture

- Cluster analysis
- Similarity and distance
- K-means clustering
- Partitioning Around Medoids (PAM) clustering
- Hierarchical clustering

Cluster analysis

- Objects belong to some groups or classes
- Cluster analysis is so-called *Unsupervised learning* of groups
- Neither the identity nor the number of groups is known
- Can be a non-parametric (algorithmic) data-driven search - today
- Or statistical grounded through semi-parametric mixture models - advanced
- Later in the course, we will return to *supervised learning* where the identities and number of groups are known (discriminant analysis and classification)

Groups in data

- A data set with n cases (rows) and p variables (columns)

$$\mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \vdots & \vdots \\ x_{n1} & \cdots & x_{np} \end{bmatrix}$$

- We look for groups in the data cases
 - Cases in the same group are «similar»
 - Cases in different groups are «different»
- Grouping or similarity is relative to some **similarity** or **distance metric**
 - What do we mean by the «distance» between two points?

Motivation

- **Explorative**
 - A way to «inspect» large data sets
 - May give you some ideas/hypotheses
 - May reveal outliers or severe errors in data
- **Down-sampling**
 - Each cluster center is a representative of its «region» in the data

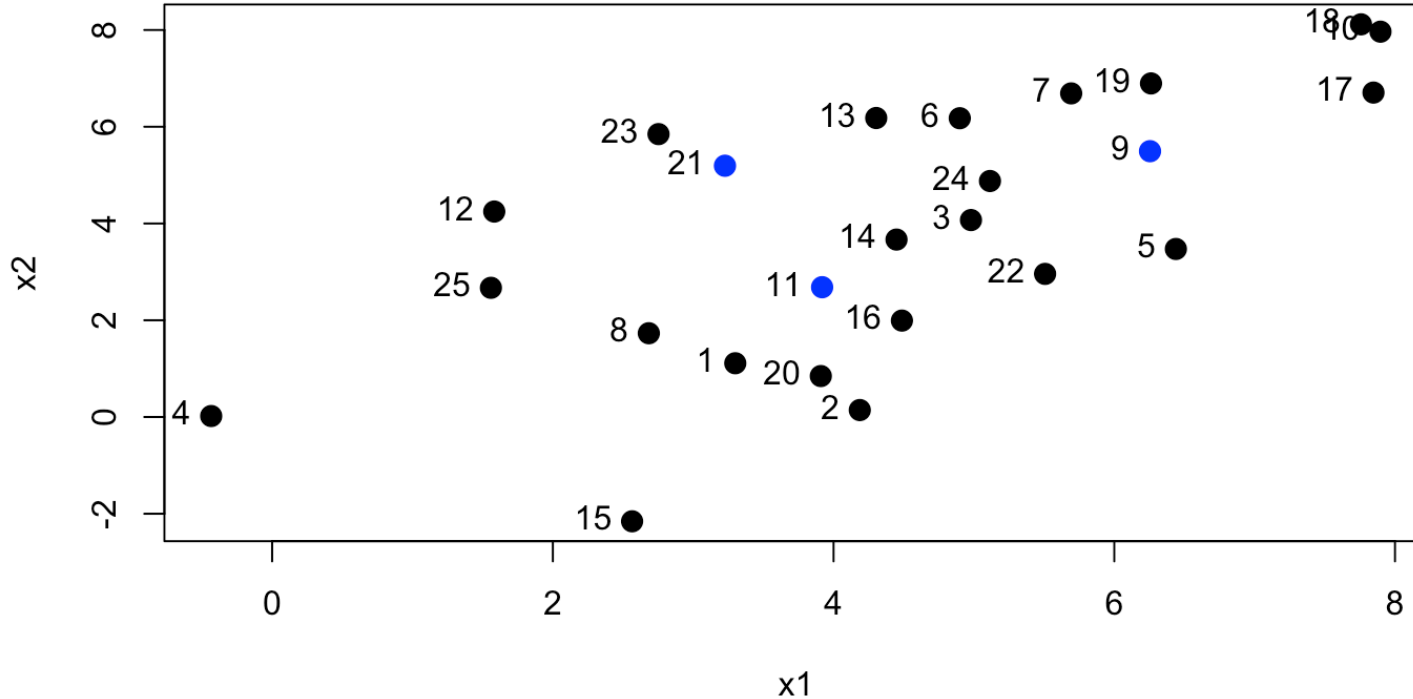
Measure of similarity/dissimilarity

We need measures of “distance” between observations

- Euclidean distance
- Manhattan distance
- Minkowski distance
- Hamming distance (for binary variables)
- Others

See e.g. `?dist` for R help

A simulated example - Distance between observations?



Euclidean distance

Euclidean distance is shortest distance on a map

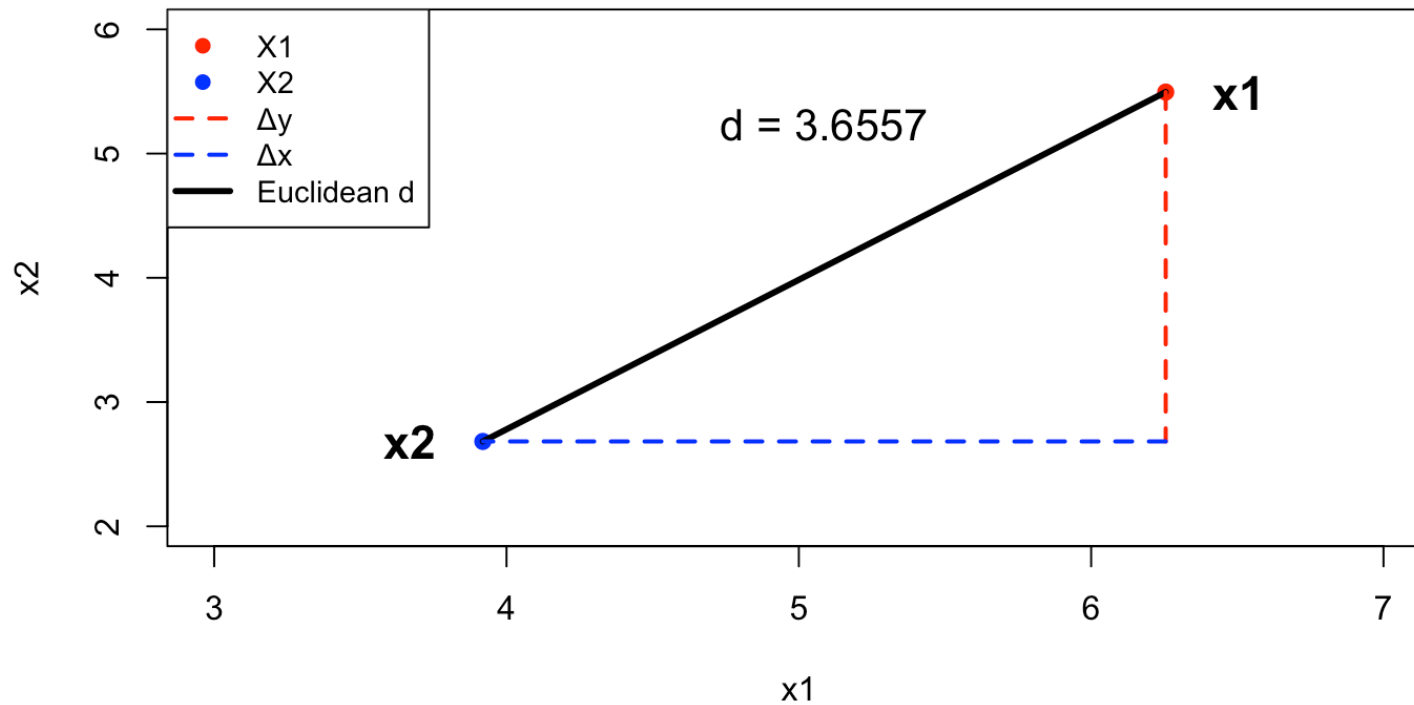
E.g. distance between two observations (x_{11}, x_{21}) and (x_{12}, x_{22})

$$D_e = \sqrt{(x_{11} - x_{12})^2 + (x_{21} - x_{22})^2}$$

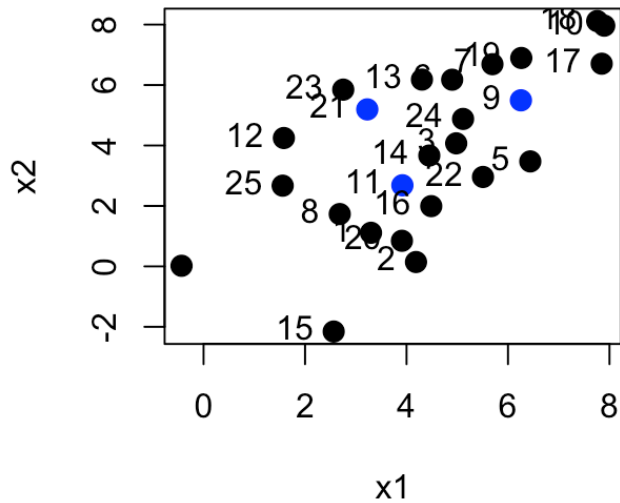
Euclidean distance: example

Assume $x_1 = (6.255, 5.495)$, $x_2 = (3.919, 2.683)$

$$d = \sqrt{(6.255 - 3.919)^2 + (5.495 - 2.683)^2}$$



For blue observations no (9, 11 and 21)

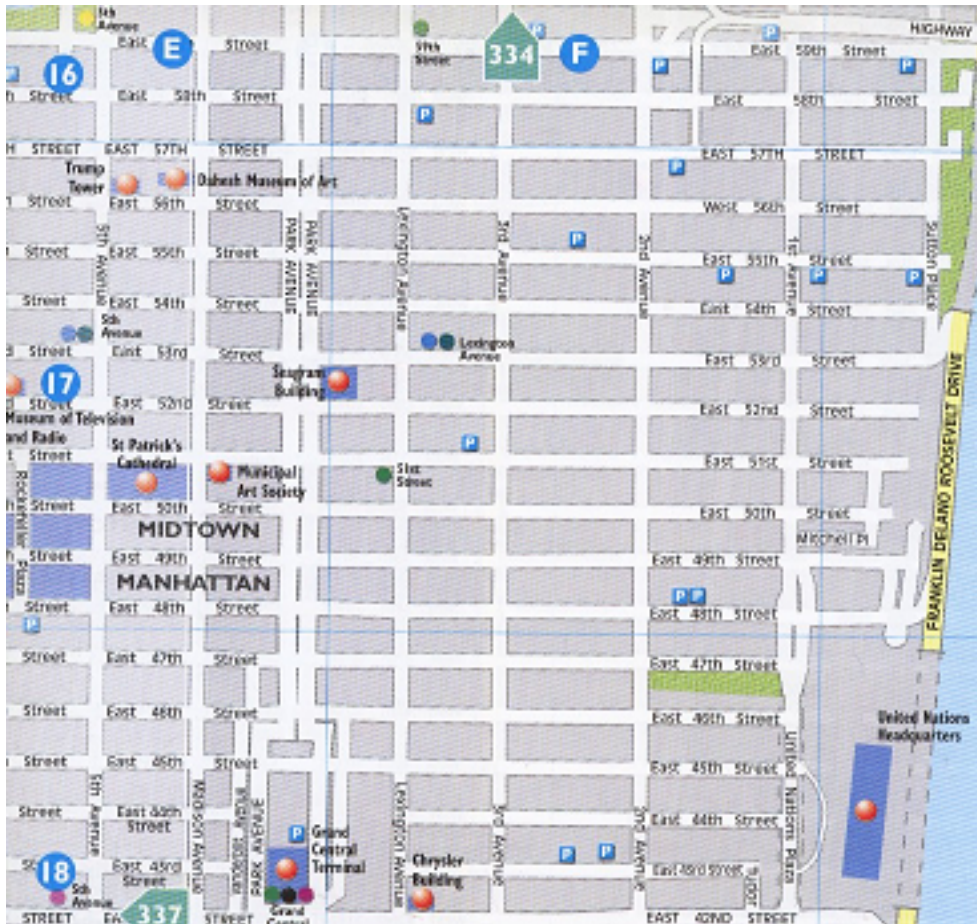


```
dist(mypoints, method = "euclidean")
```

```
##           9           11
## 11 3.655337
## 21 3.043111 2.604636
```

Manhattan distance

“Manhattan distance” is like the walking distance between two shops on Manhattan



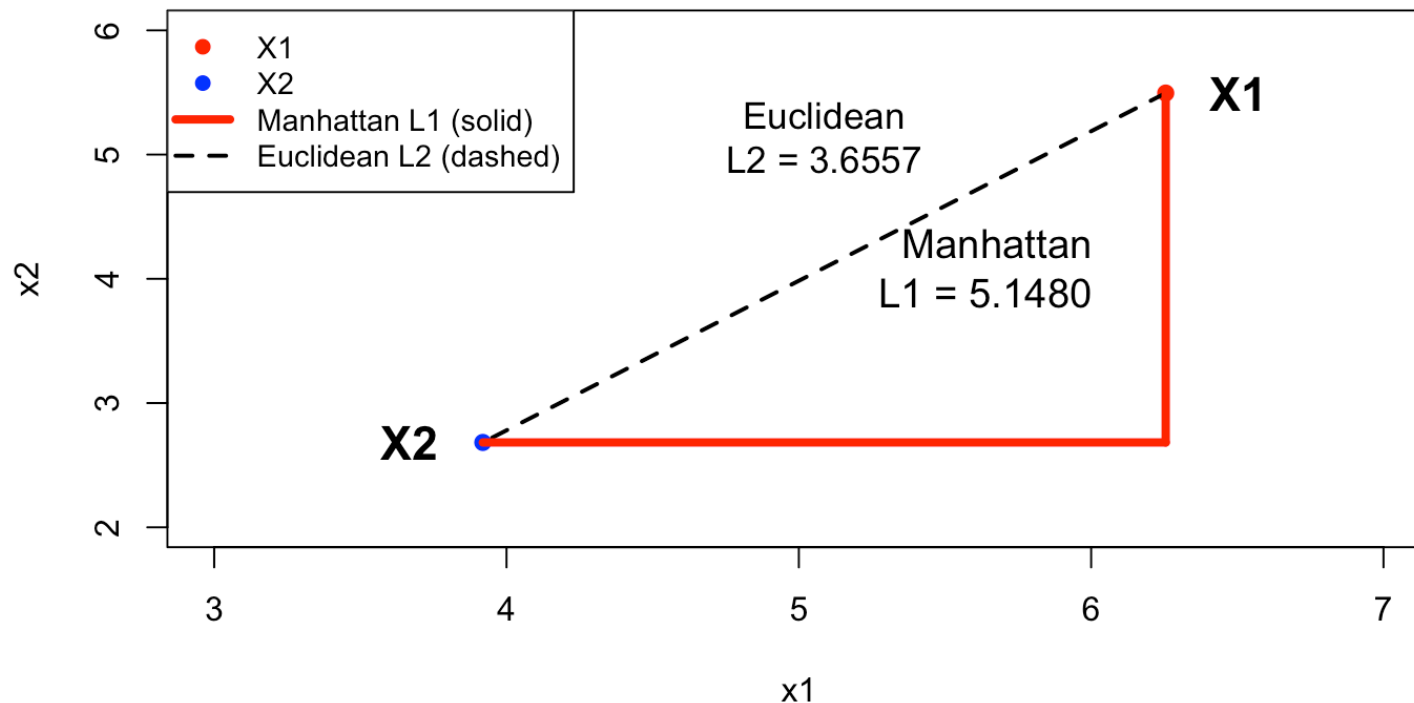
Manhattan distance, continued

Manhattan distance between two observations

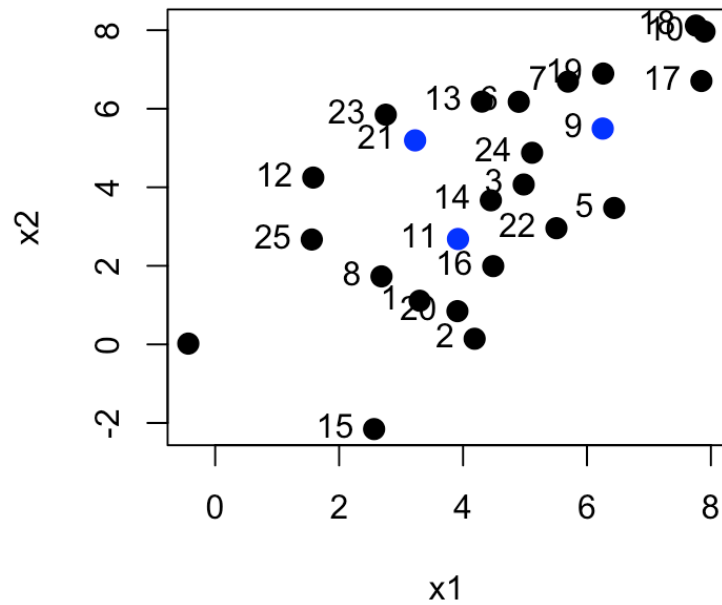
$$x_1 = (6.255, 5.495), x_2 = (3.919, 2.683)$$

$$D_m = |x_{11} - x_{12}| + |x_{21} - x_{22}|$$

`manh_dist = abs(X1[1] - X2[1]) + abs(X1[2] - X2[2])`



Manhattan distance for blue points:



```
##          9          11
## 11 5.147513
## 21 3.328770 3.203190
```

Minkowski distance

For two points $\mathbf{x}_1 = (x_{11}, \dots, x_{1p})$ and $\mathbf{x}_2 = (x_{21}, \dots, x_{2p})$, the Minkowski distance of order $r \geq 1$ is

$$d_r(\mathbf{x}_1, \mathbf{x}_2) = \left(\sum_{j=1}^p |x_{1j} - x_{2j}|^r \right)^{1/r}.$$

- Manhattan distance ($r = 1$): $d_1(\mathbf{x}_1, \mathbf{x}_2) = \sum_{j=1}^p |x_{1j} - x_{2j}|$
- Euclidean distance ($r = 2$): $d_2(\mathbf{x}_1, \mathbf{x}_2) = \sqrt{\sum_{j=1}^p (x_{1j} - x_{2j})^2}$
- Larger r values give more weight to large coordinate differences.

Hamming distance

For two **binary** vectors $\mathbf{x}_1, \mathbf{x}_2 \in \{0, 1\}^p$ of the same length, the **Hamming distance** counts how many positions differ:

$$d_H(\mathbf{x}_1, \mathbf{x}_2) = \#\{j : x_{1j} \neq x_{2j}\}.$$

Example (differences highlighted in red):

$$\mathbf{x}_1 = (\textcolor{red}{1}, 0, \textcolor{red}{1}, 0), \quad \mathbf{x}_2 = (\textcolor{red}{0}, 0, \textcolor{red}{0}, 0)$$

Positions 1 and 3 differ, so $d_H(\mathbf{x}_1, \mathbf{x}_2) = 2$.

- Used for **categorical** or **binary** variables (e.g., gene sequences, binary features).
- Related to the number of bit flips needed to transform one vector into the other.

Iris data

Goal: Group together objects that are similar



Iris versicolor



Iris virginica



The famous “Iris” data set

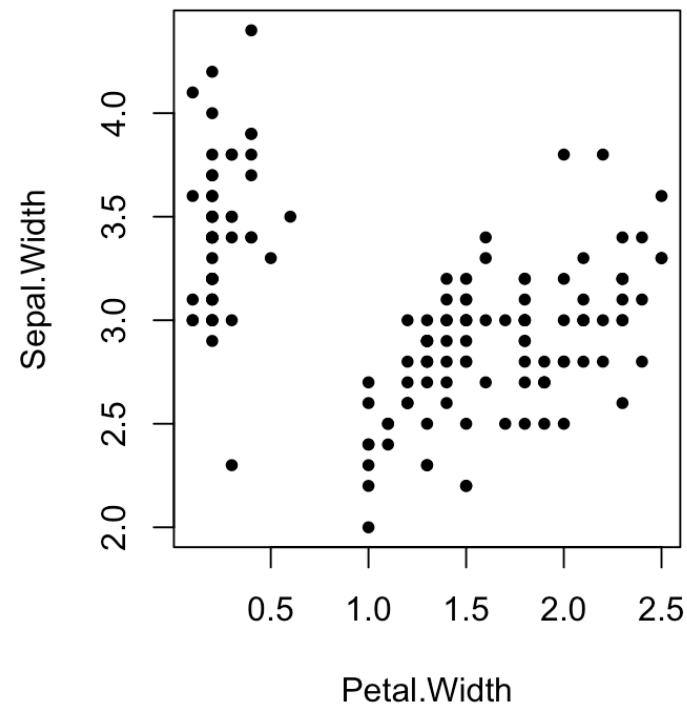
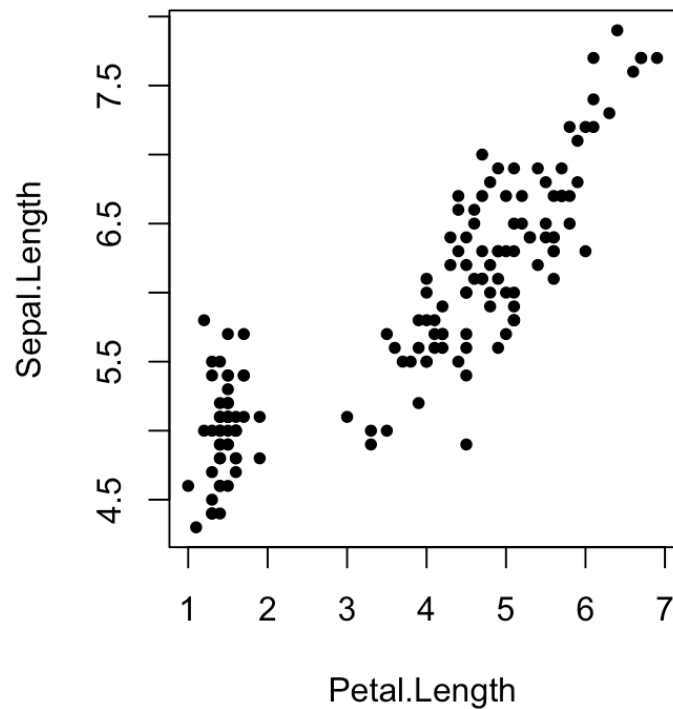
- This famous (Fisher’s or Anderson’s) iris data set gives the measurements in centimeters of the variables X1=sepal length, X2=sepal width, X3=petal length and X4=petal width, respectively.

##	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
## 1	5.1	3.5	1.4	0.2	setosa
## 2	4.9	3.0	1.4	0.2	setosa
## 3	4.7	3.2	1.3	0.2	setosa
## 55	6.5	2.8	4.6	1.5	versicolor
## 56	5.7	2.8	4.5	1.3	versicolor
## 57	6.3	3.3	4.7	1.6	versicolor
## 125	6.7	3.3	5.7	2.1	virginica
## 126	7.2	3.2	6.0	1.8	virginica

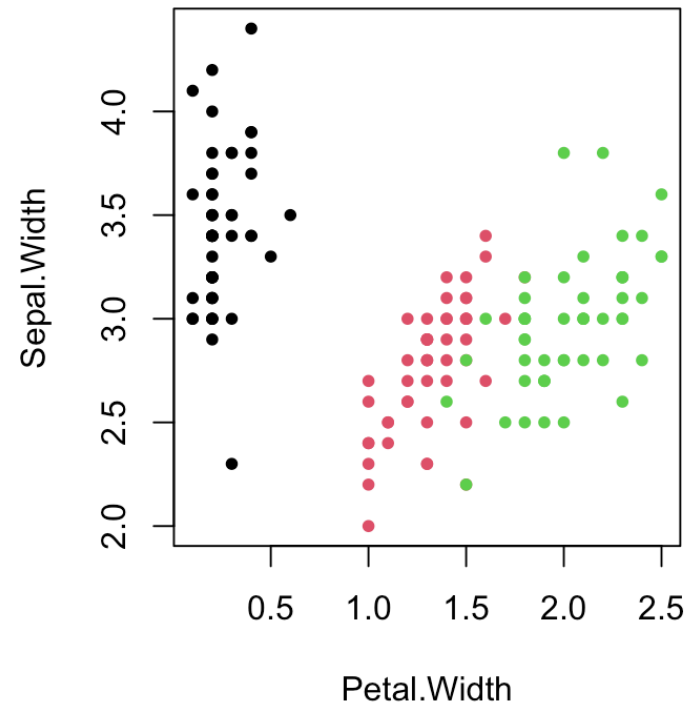
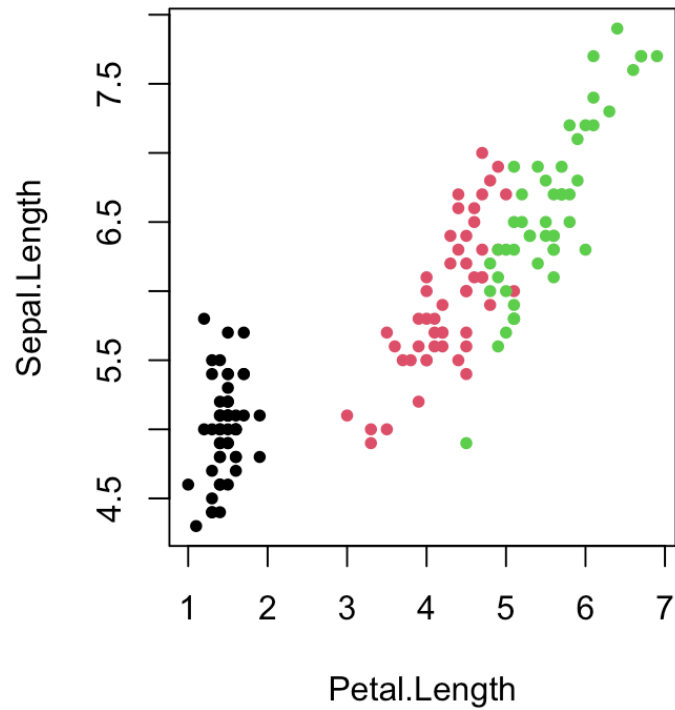
- Question: Are there many species of iris? which are which? (if we did not know the truth)

Sepal vs petal length/width

- Are there detectable clusters in the data?



For these data we know...



K-means clustering

- Distribute the observations into K clusters so that the **within-cluster** dissimilarity (from **Euclidean distance**) is minimized.
- For cluster C_k , the within-cluster sum of squared distances is

$$W(C_k) = \sum_{\mathbf{x}_i \in C_k} d^2(\mathbf{x}_i, \boldsymbol{\mu}_k),$$

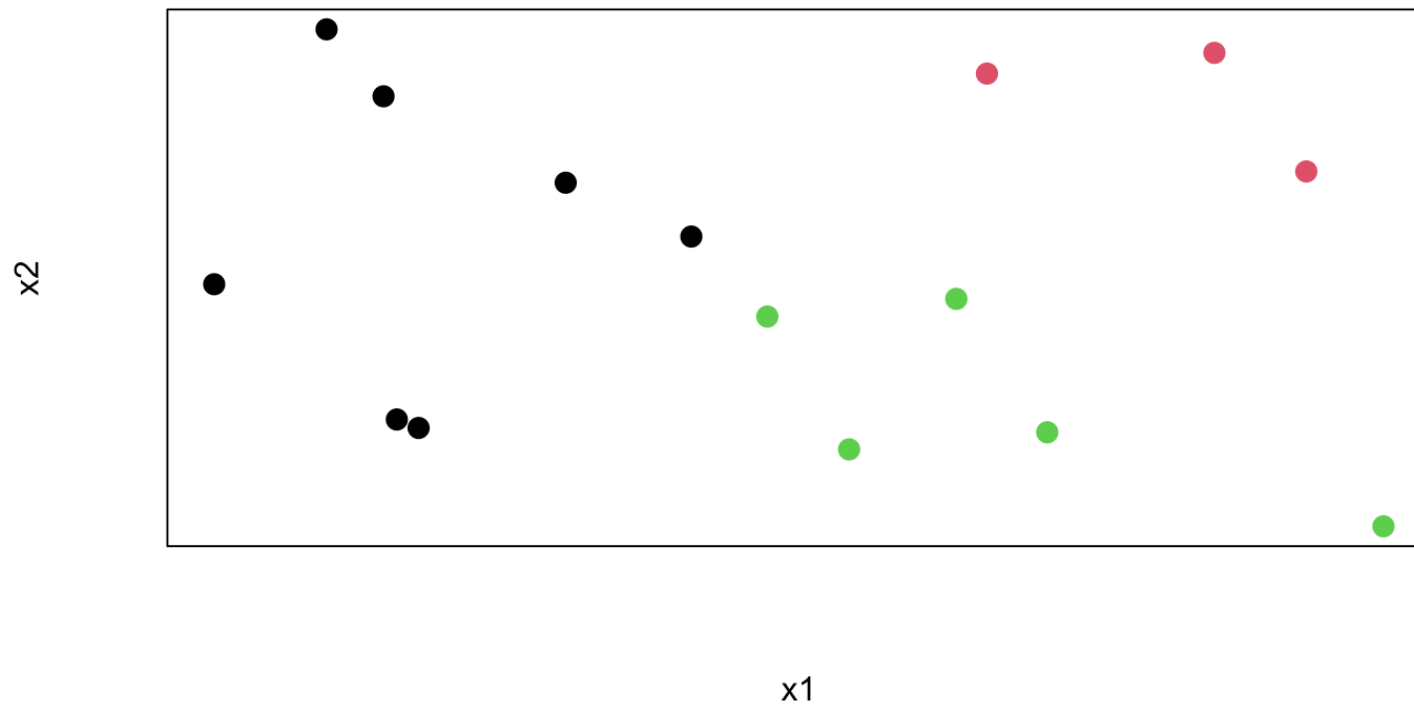
where $d(\cdot, \cdot)$ is the distance (here Euclidean) and $\boldsymbol{\mu}_k$ is the centroid of C_k .

- The k-means objective is to find clusters $C_1, \dots, C_K$ such that

$$\text{WCSS}_K = \sum_{k=1}^K W(C_k) \rightarrow \min.$$

K-means illustrated, I

- Two variables, x_1 and x_2 , and $K = 3$ clusters

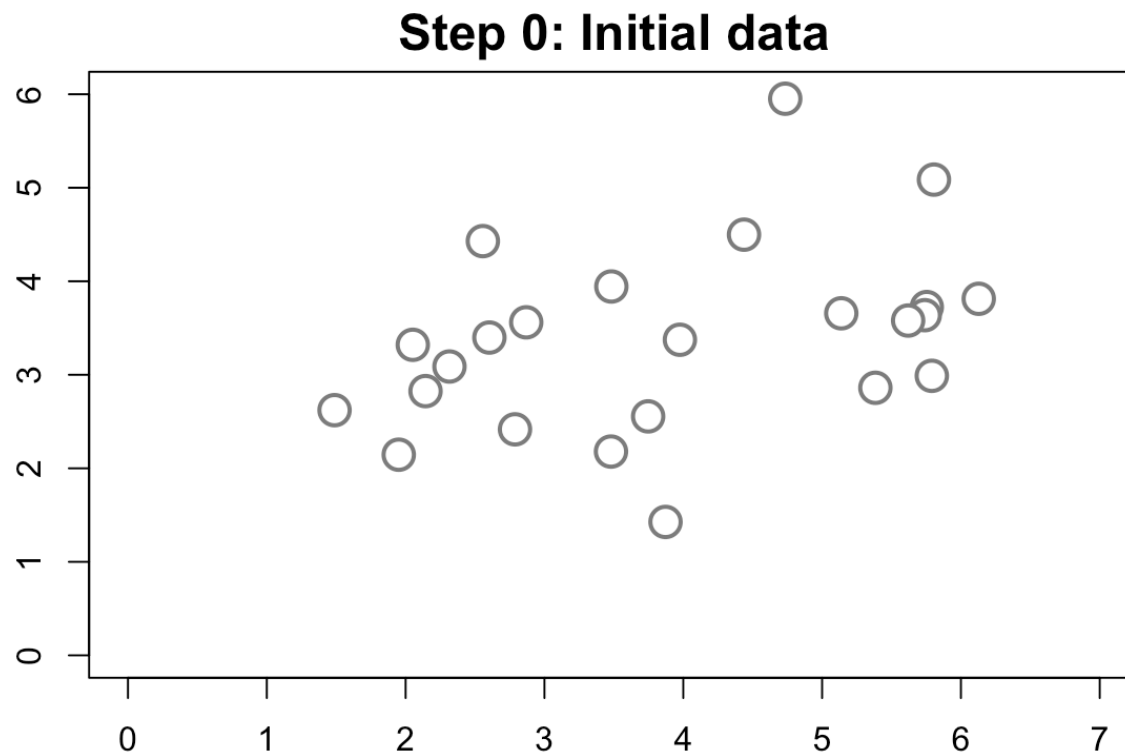


K-means clustering algorithm

1. Choose the number of clusters K and select K initial centroids (typically random observations).
 2. **Assignment step:** Assign each observation to the cluster with the closest centroid (Euclidean distance).
 3. **Update step:** Recompute each centroid as the mean of the observations currently in that cluster.
 4. Repeat steps 2–3 until the cluster assignments no longer change.
- Sensitive to the random starting centroids → may converge to a **local** minimum; therefore run k-means several times and keep the best solution.

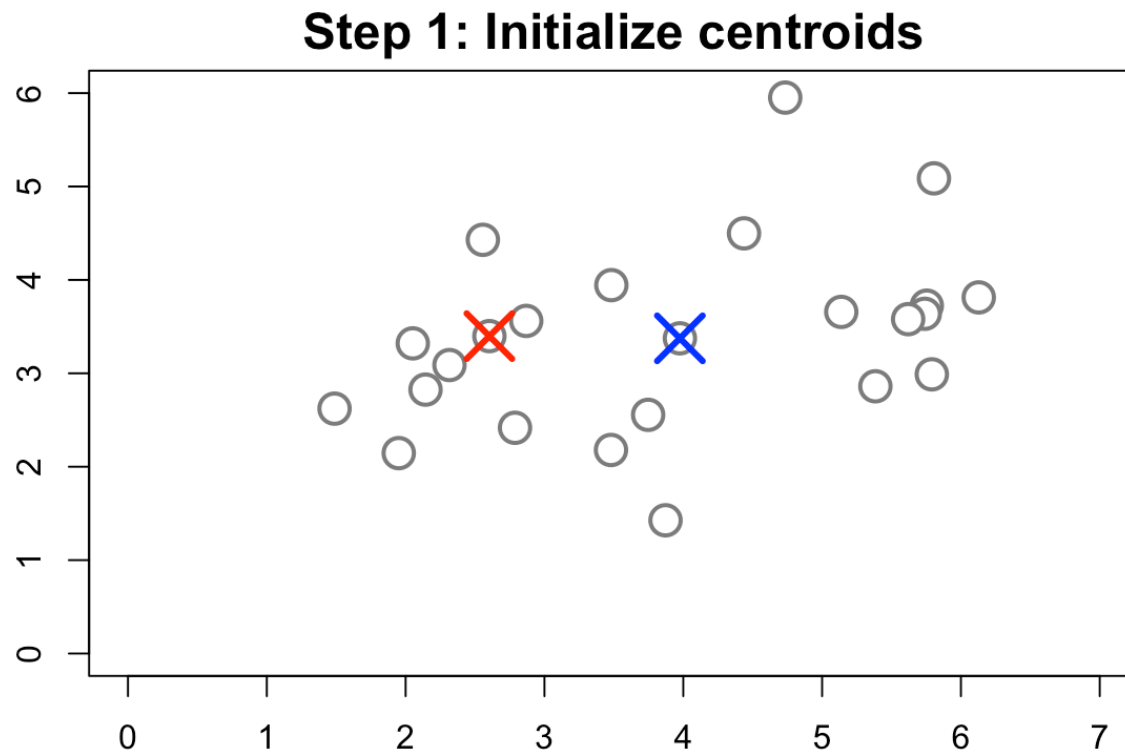
K-means: Step 0

Initial data, no clusters yet.



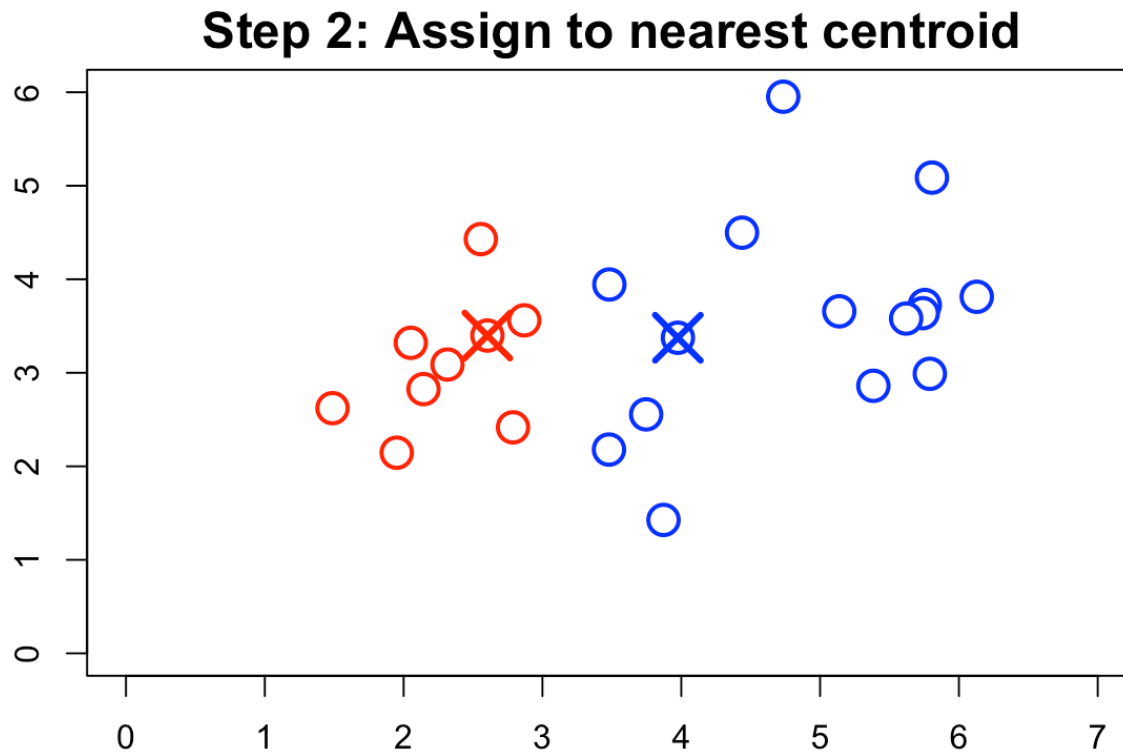
K-means: Step 1

Initialize: Randomly place $K = 2$ centroids.



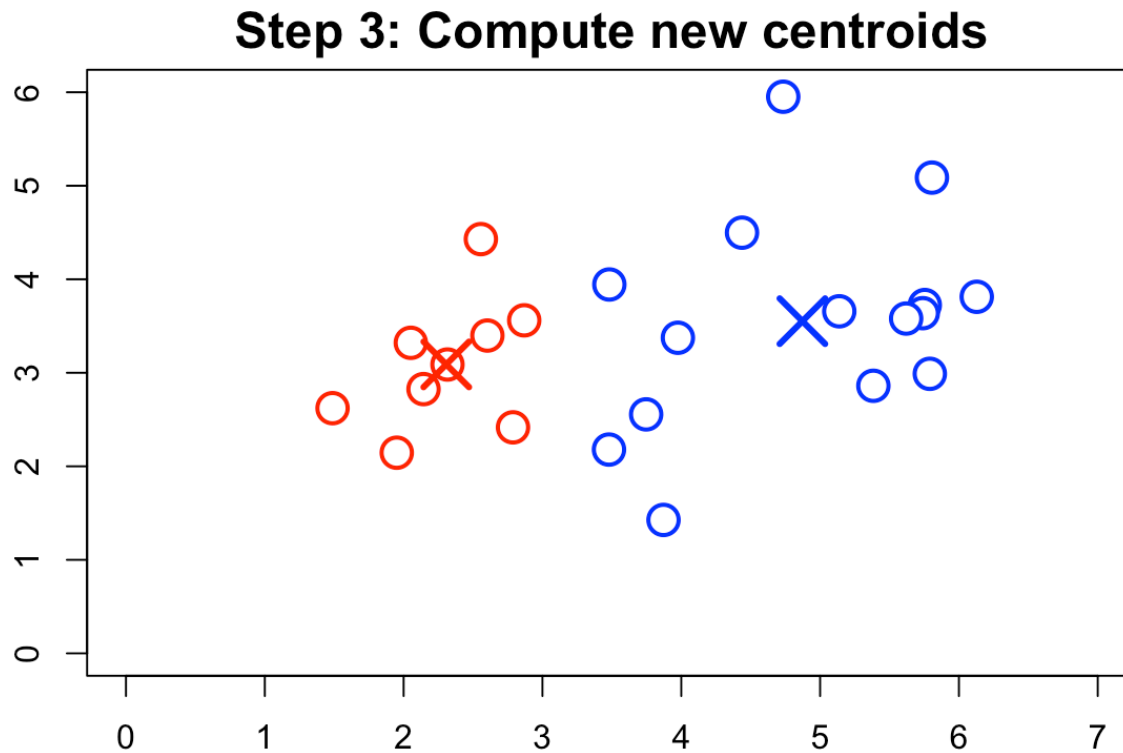
K-means: Step 2

Assign each observation to the nearest centroid.



K-means: Step 3

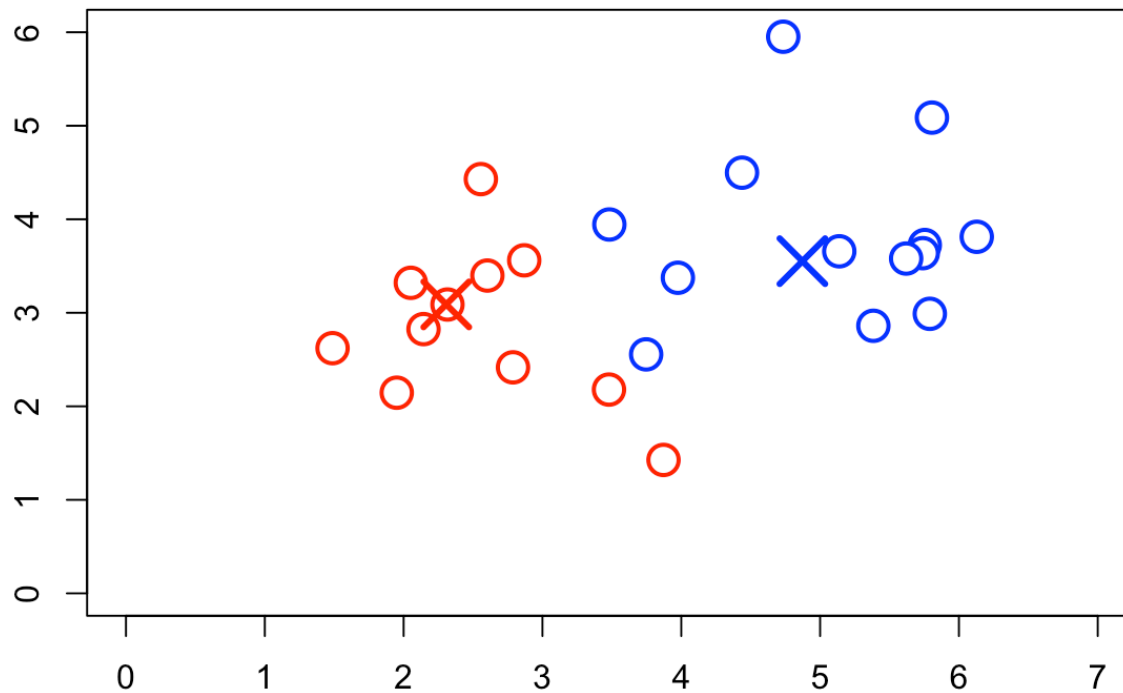
Update: Compute new centroids as the mean of assigned points.



K-means: Step 2 again

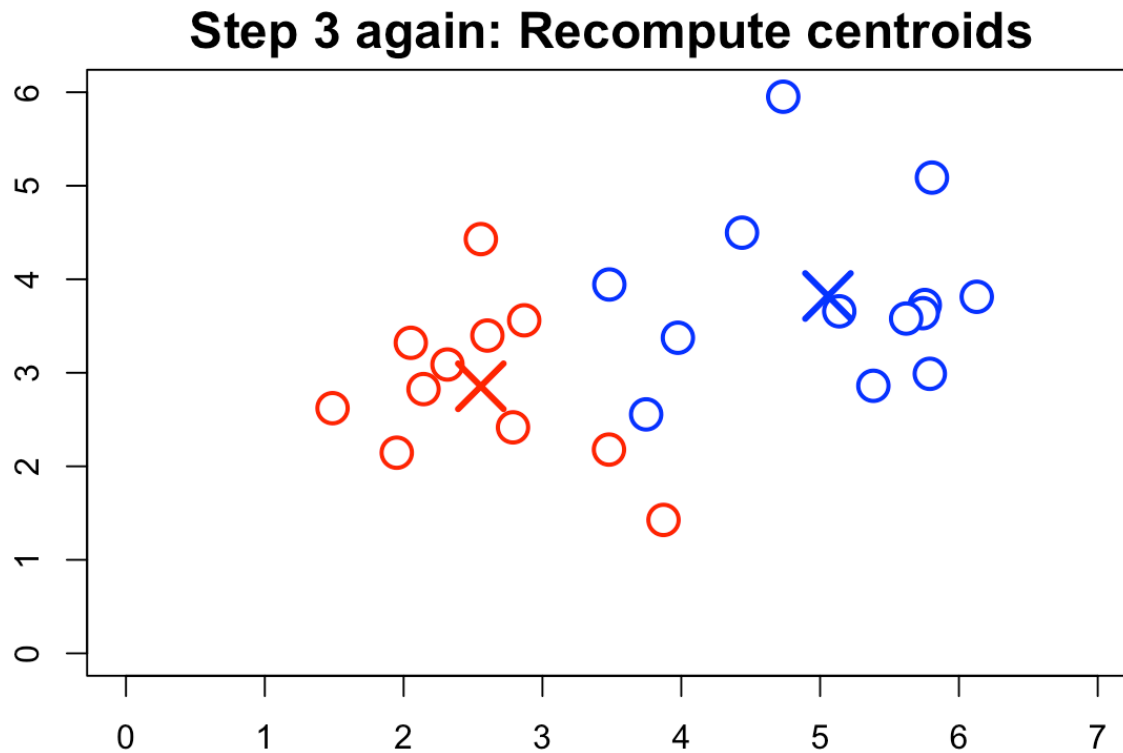
Repeat Step 2 with updated centroids. If nothing changes we are done

Step 2 again: Reassign



K-means: Step 3 again

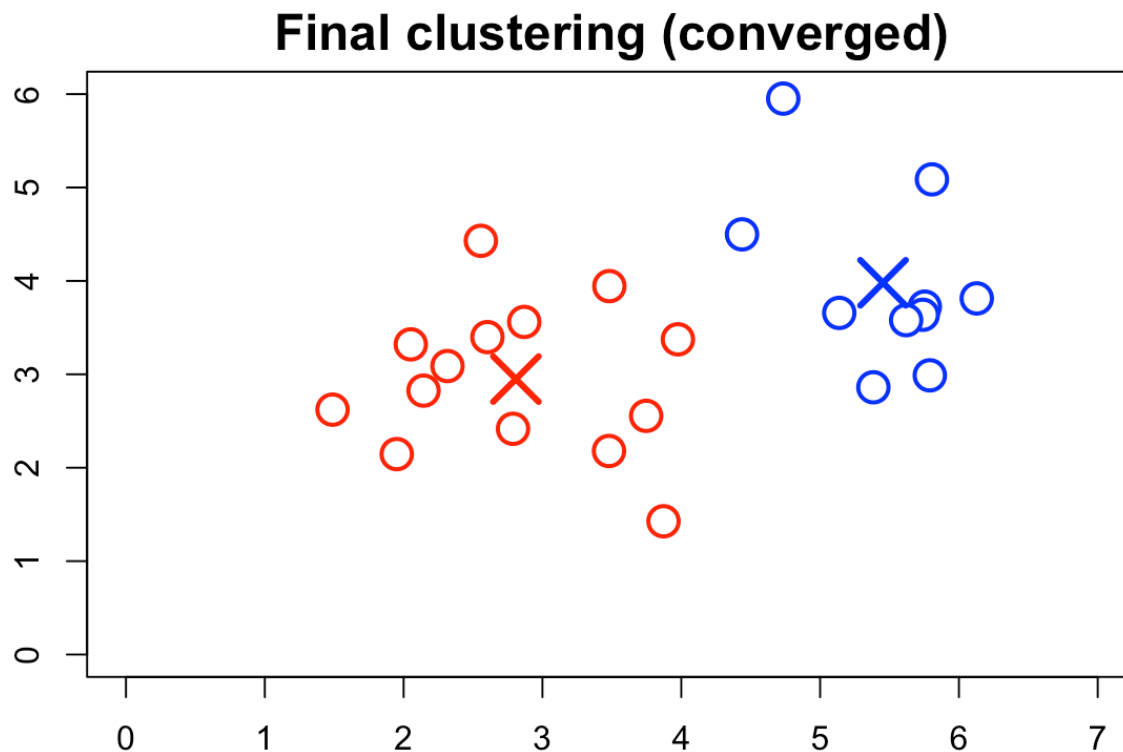
Repeat Step 3: recompute centroids.



K-means: Convergence

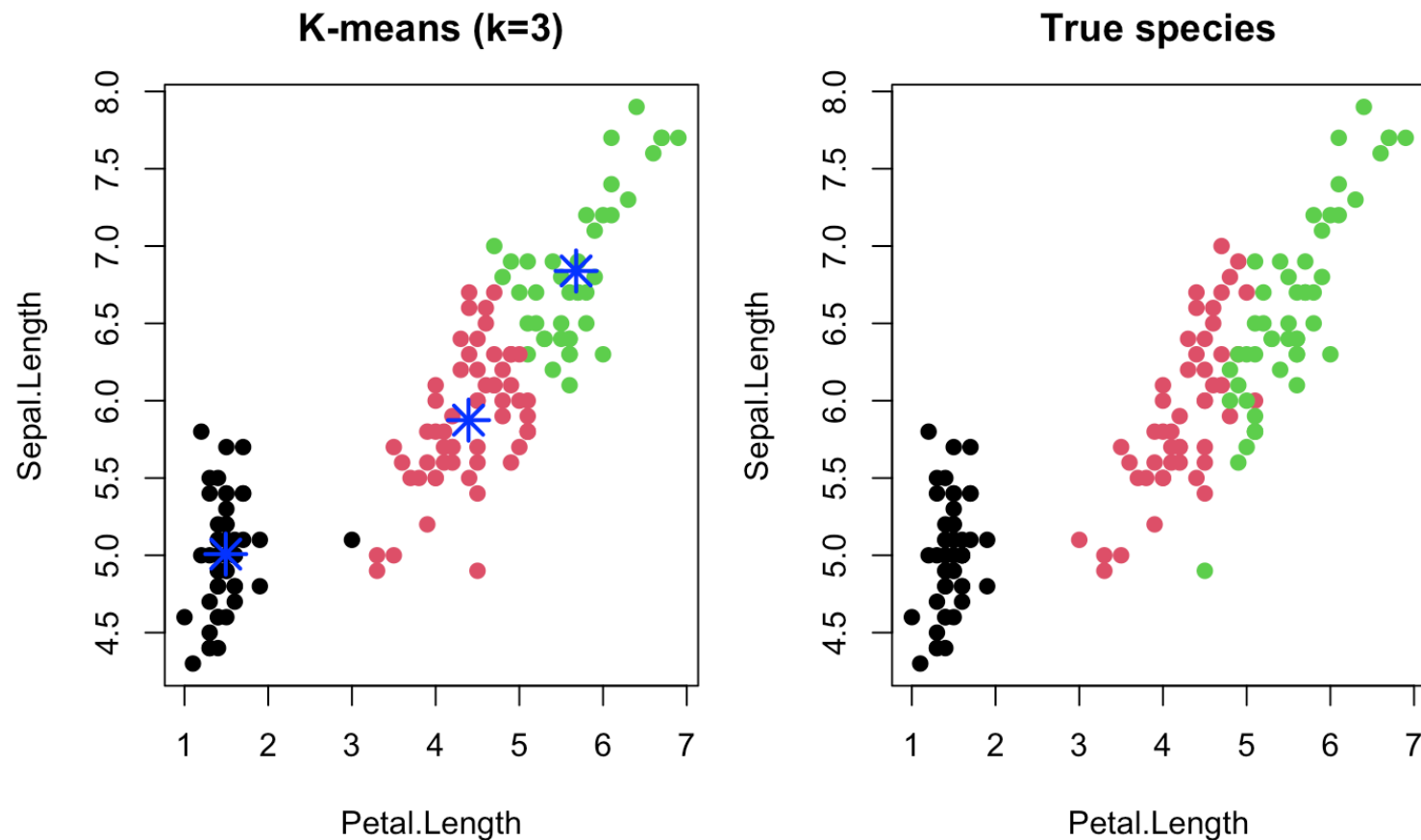
If assignments not changing, the algorithm **converged**. In R:

```
km_final <- kmeans(X, centers = 2, nstart = 100)
```



K-means on the iris data (R-code)

```
df <- data.frame(x=iris[,3], y=iris[,1])  
kmeans_result <- kmeans(df, centers = 3, nstart = 100)
```



K-means sums of squares

```
kmeans_result$size # Cluster Sizes
```

```
## [1] 51 41 58
```

```
kmeans_result$withinss # Within Cluster Sum of Squares
```

```
## [1] 9.893725 20.407805 23.508448
```

```
kmeans_result$tot.withinss # Total Within Sum of Squares
```

```
## [1] 53.80998
```

Interpretation: - **size:** number of observations in each cluster -
withinss: sum of squared distances within each cluster - **tot.withinss:**
total variation within clusters (lower is better)

K-means on the full iris data (R-code)

```
df <- iris[,1:4]  
kmeans_result <- kmeans(df, centers = 3, nstart = 100)
```

K-means sums of squares

```
kmeans_result$size          # Cluster Sizes
```

```
## [1] 62 38 50
```

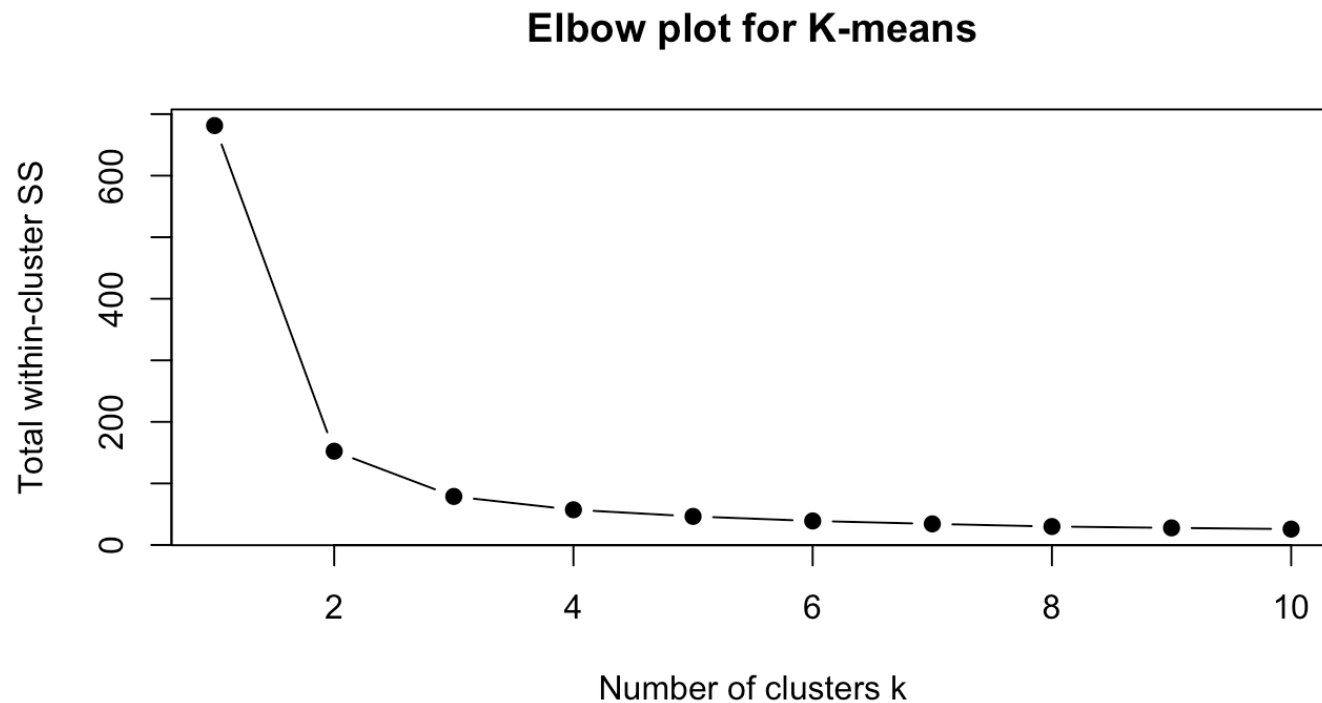
```
kmeans_result$withinss      # Within Cluster Sum of Squares
```

```
## [1] 39.82097 23.87947 15.15100
```

```
kmeans_result$tot.withinss  # Total Within Sum of Squares
```

```
## [1] 78.85144
```

Choosing K: Elbow Plot



Look for the “elbow” where adding more clusters gives diminishing returns.

AIC and BIC for K-means

$WCSS_K = \text{km\$tot.withinss}$ be the residual sum of squares and $\hat{\sigma}_K^2 = WCSS_K/n$ the variance.

Number of parameters: $\text{params}_K = K \times p + 1$ (K of p -dimensional cluster means plus one common variance). Then

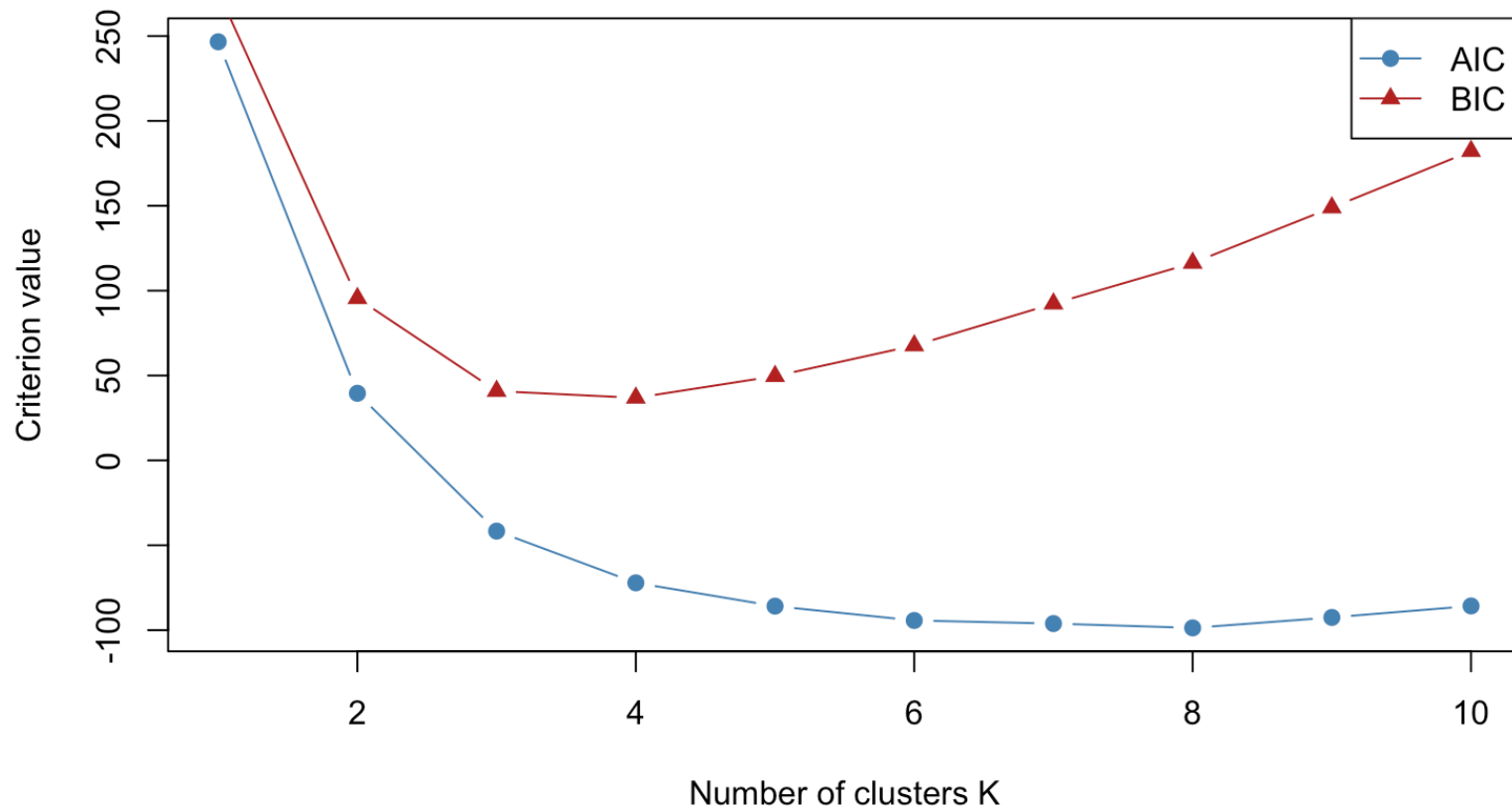
$$\text{AIC}(K) = n \log \left(\frac{WCSS_K}{n} \right) + 2 \text{params}_K ,$$

$$\text{BIC}(K) = n \log \left(\frac{WCSS_K}{n} \right) + \log(n) \text{params}_K .$$

- The term $n \log(WCSS_K/n)$ plays the role of a (negative) log-likelihood based on the residual sum of squares.
- The penalty terms 2params_K (AIC) and $\log(n) \text{params}_K$ (BIC) discourage using too many clusters.

Choosing K: AIC and BIC

AIC / BIC for k-means (Gaussian approximation)



Partitioning Around Medoids (PAM)

- The `kmeans()` function in always uses **Euclidean** distances.
- The `pam()` function (package `cluster`) is similar to k-means, but
 - Uses **medoids** instead of centroids (an actual data point closest to the center).
 - You can supply any distance **matrix** here.

$$W_k = \sum_{i \in C_k} d(\mathbf{x}_i, \mathbf{m}_k) \quad \text{for cluster } C_k$$

Find clusters $C_1, \dots, C_K$ such that $\sum_{k=1}^K W_k \rightarrow \min$

PAM with custom distance matrix

PAM can work with any dissimilarity matrix, not just Euclidean distance.

```
library(cluster)
set.seed(123)
```

```
# Use iris data
```

```
df <- iris[, c(3, 1)] # Petal.Length, Sepal.Length
```

```
pam_manhattan <- pam(df, k = 3, metric = "manhattan", nstart = 100)
```

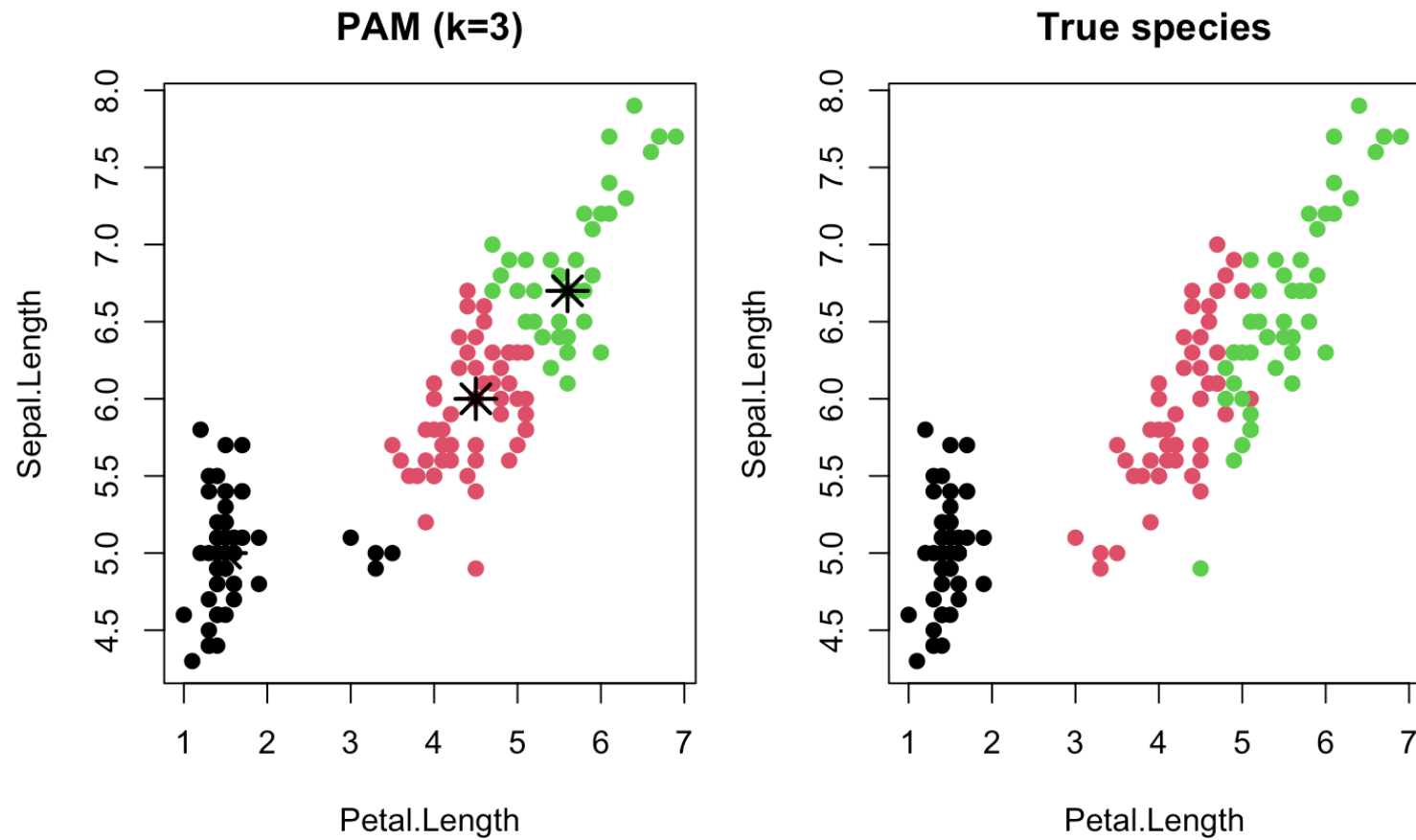
```
# Compute Minkowski distance matrix
```

```
dist_minkowski <- dist(df, method = "minkowski")
```

```
# Run PAM on the distance matrix
```

```
pam_minkowski <- pam(dist_minkowski, k = 3, diss = TRUE, nstart = 100)
```

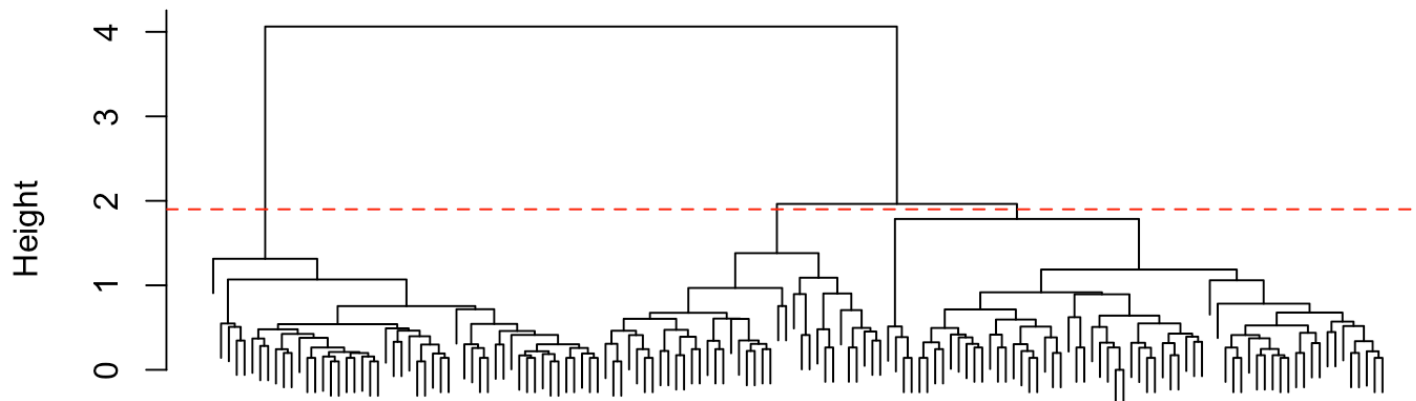
PAM clustering on iris data



Hierarchical clustering

- Builds a hierarchical distance structure – dendrogram (tree)
 - We often plot this dendrogram
- Clustering into distinct groups by cutting the dendrogram
- Requires we define distance between clusters (linkage function)

Hierarchical clustering of iris data



Hierarchical clustering algorithm

Each of the n observations is a point in a p -dimensional space

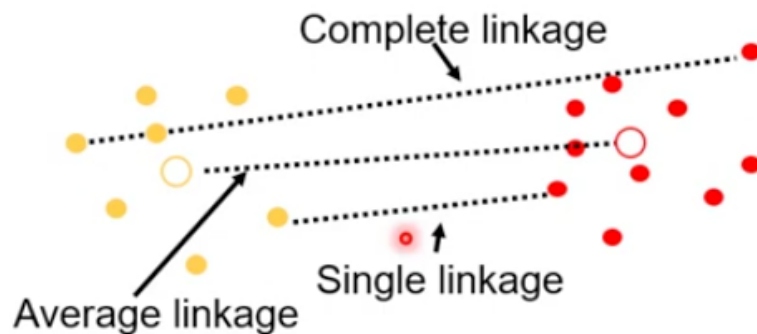
Agglomerative (bottom-up) algorithm to build a dendrogram:

1. **Initialize:** Start with n clusters, with single observations.
2. **Compute distances:** Calculate the distance between every pair of clusters using a chosen **distance metric**.
3. **Merge:** Find the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distances:** Recompute distances between the new cluster and all remaining clusters using a **linkage function**
5. **Repeat:** Go back to step 3 until all observations belong to a single cluster.

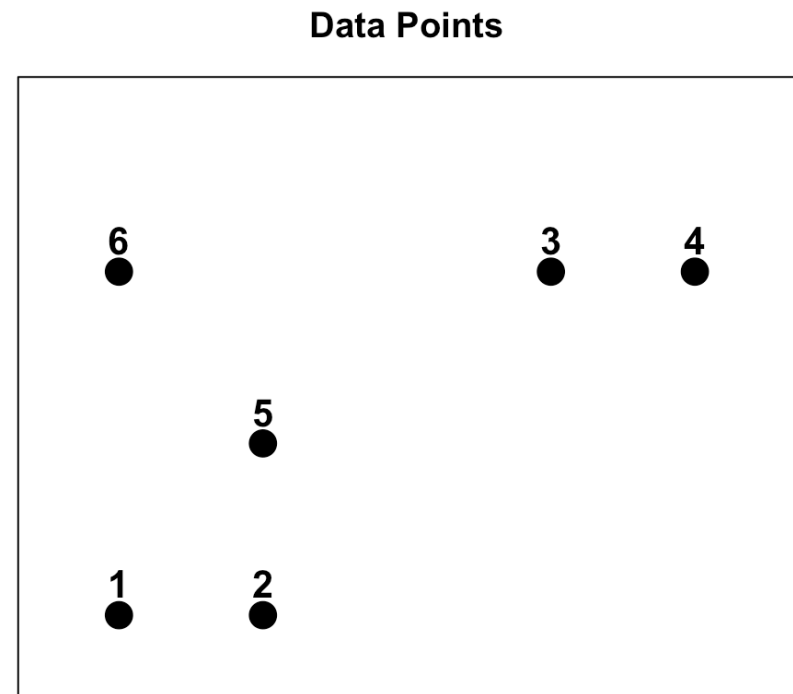
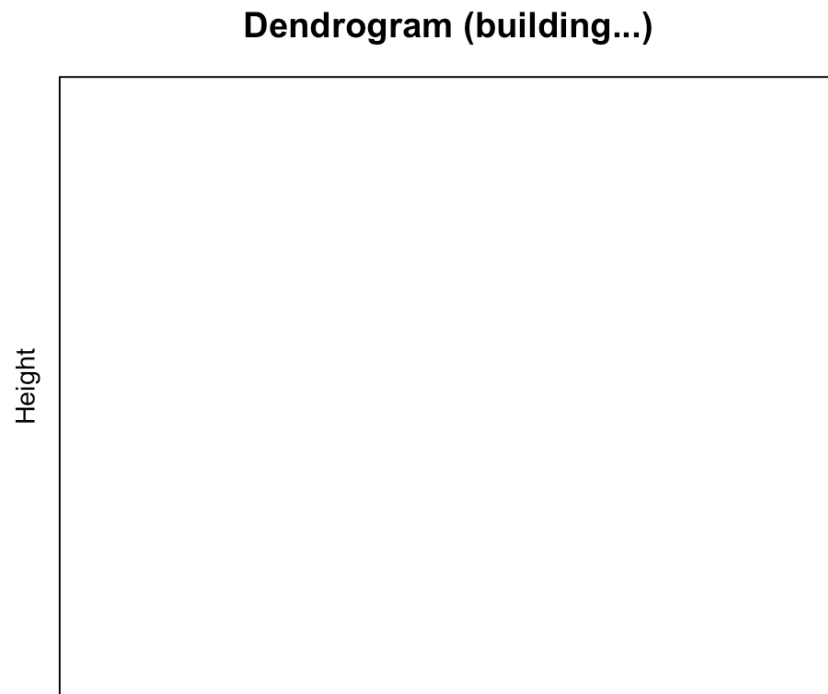
Agglomeration method

The three common distance measures between groups are:

1. Single: Distance between groups = distance between closest group members
2. Average: Distance between groups = distance between group average points
3. Complete: Distance between groups = distance between most distant group members



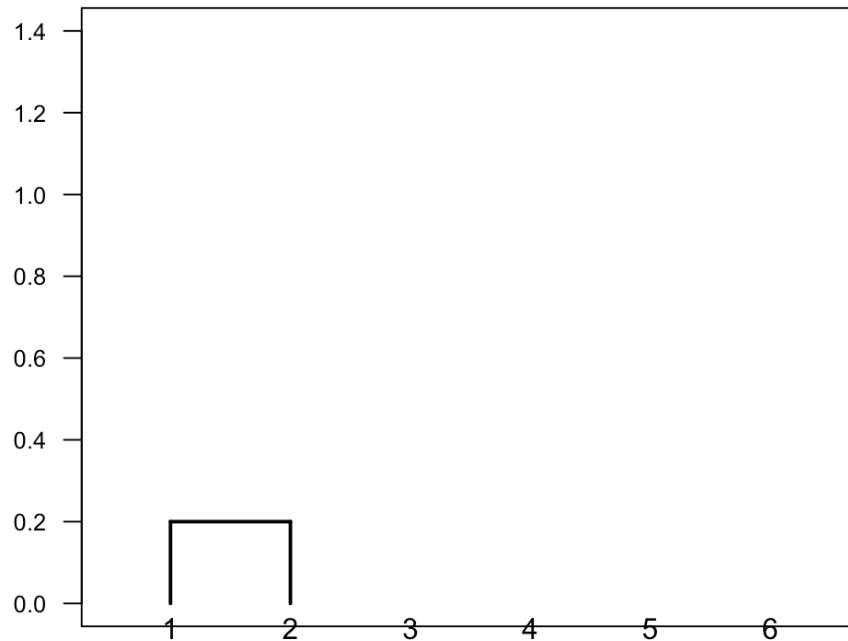
Single Linkage: Initial Points



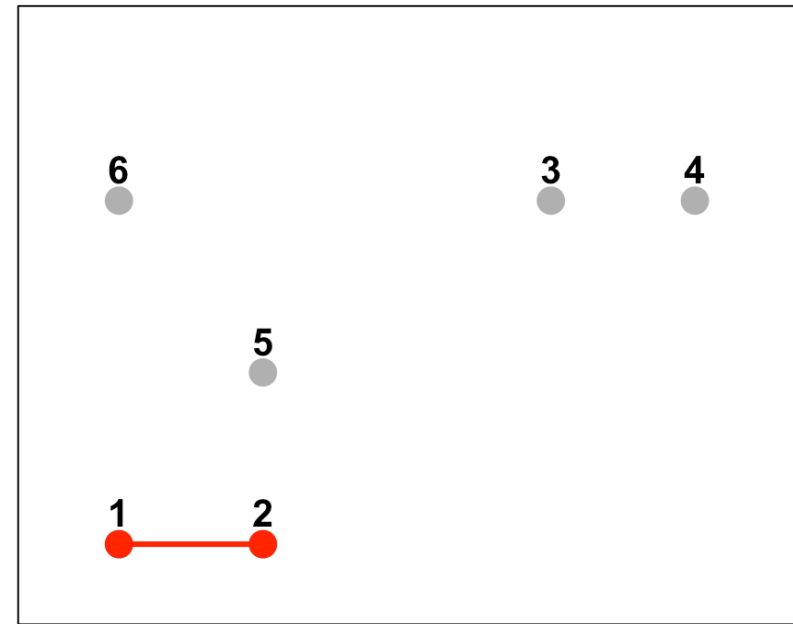
6 clusters (each point is its own cluster)

Single Linkage: Step 1 (merge 1 & 2)

Single Linkage: 1st merge



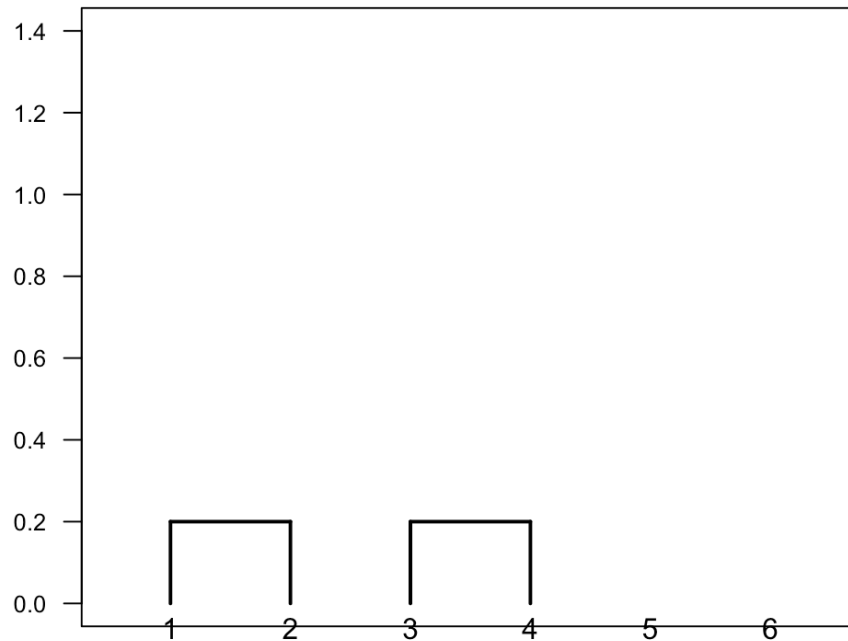
Merge points 1 & 2



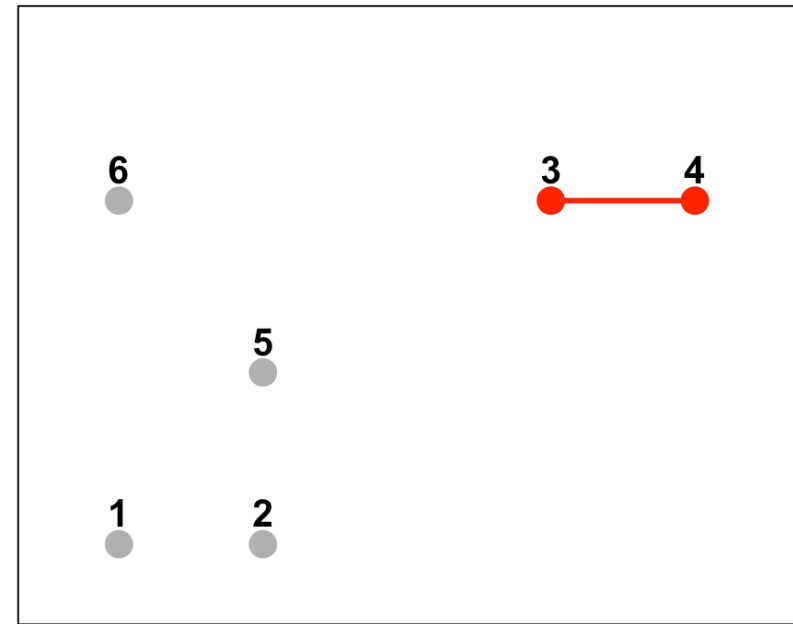
Closest pair: Points 1 & 2 (distance = 0.20) → 5 clusters

Single Linkage: Step 2 (merge 3 & 4)

Single Linkage: 2nd merge



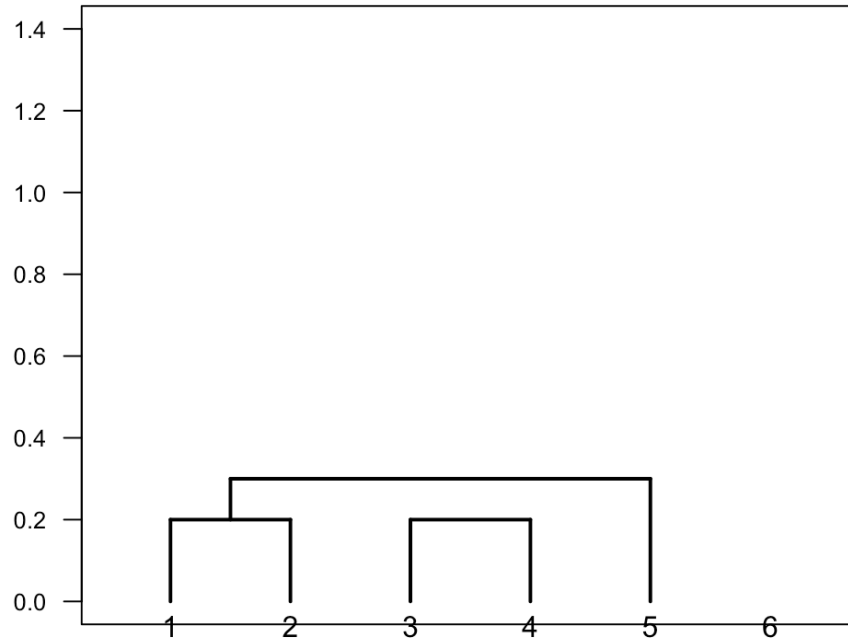
Merge points 3 & 4



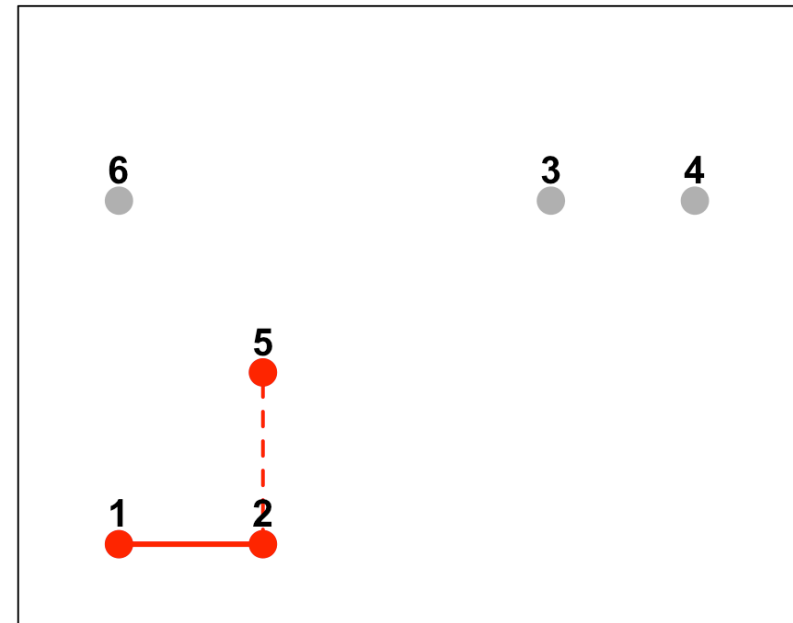
Next closest pair: Points 3 & 4 (distance = 0.20) → 4 clusters: {1,2}, {3,4}, {5}, {6}

Single Linkage: Step 3 (5 joins {1,2})

Single Linkage: 3rd merge



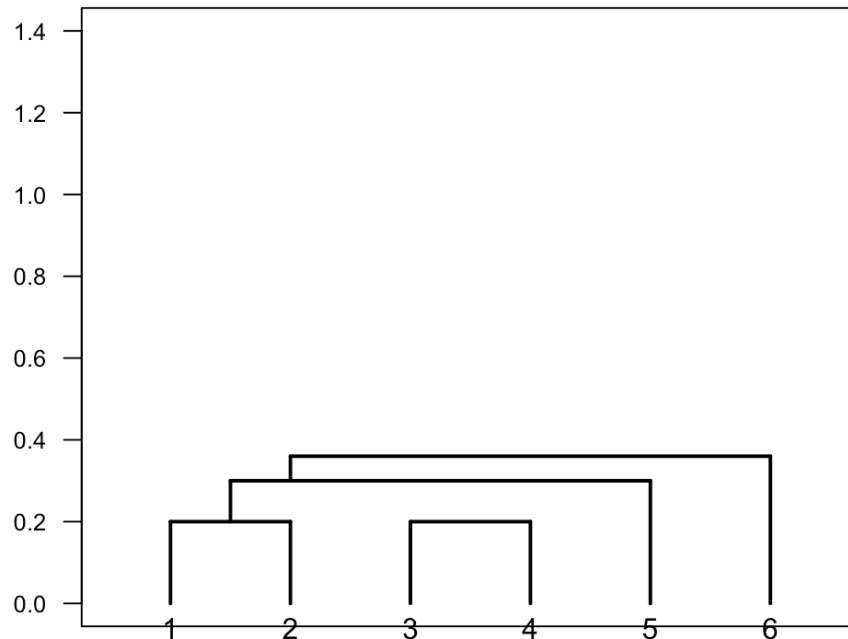
Point 5 joins cluster {1,2}



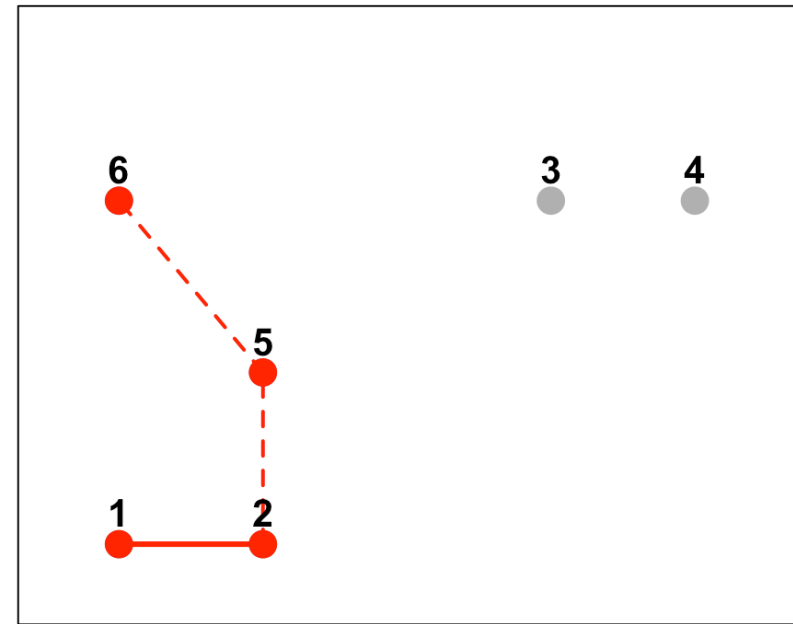
Point 5 joins {1,2} at height $\approx 0.30 \rightarrow$ 3 clusters: {1,2,5}, {3,4}, {6}

Single Linkage: Step 4 (6 joins {1,2,5})

Single Linkage: 4th merge

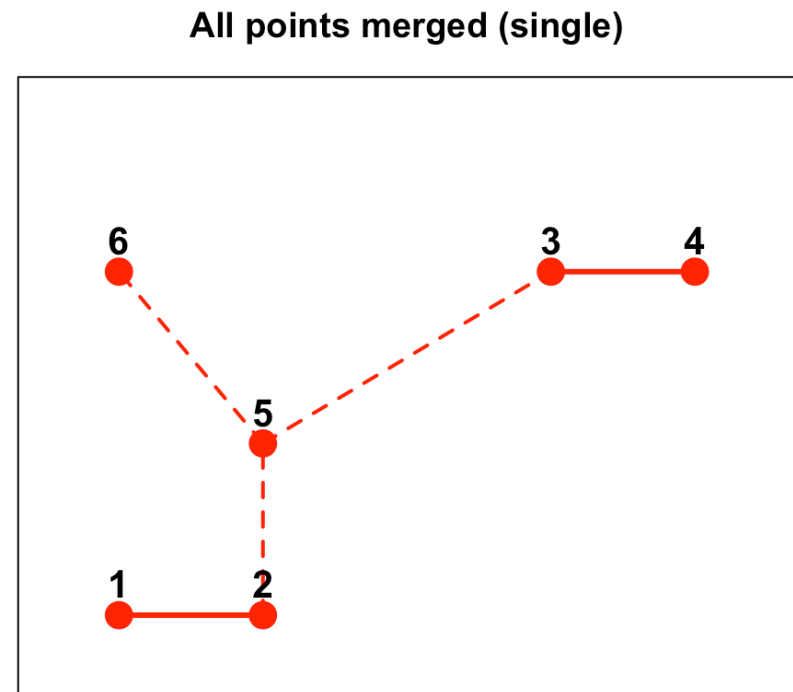
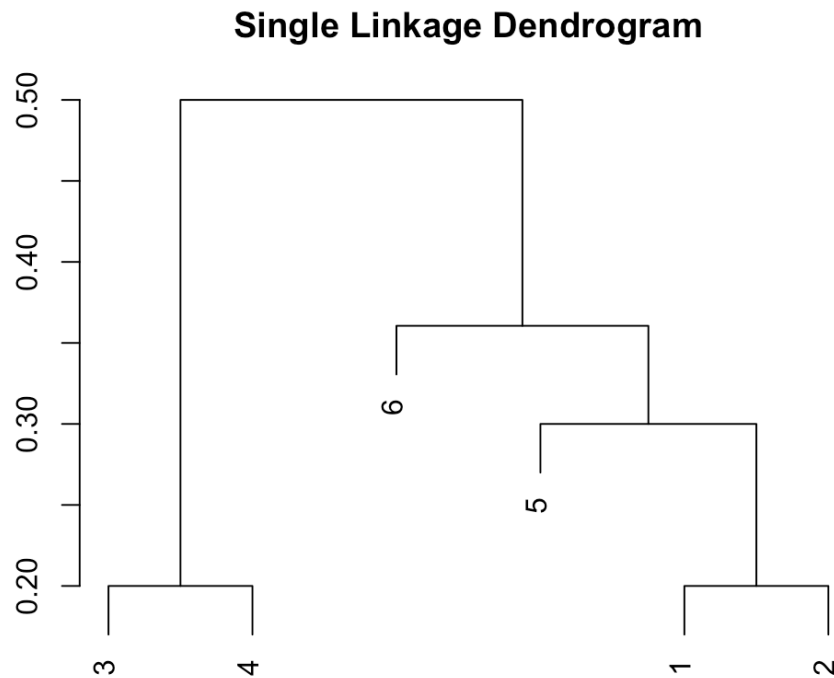


Point 6 joins cluster {1,2,5}



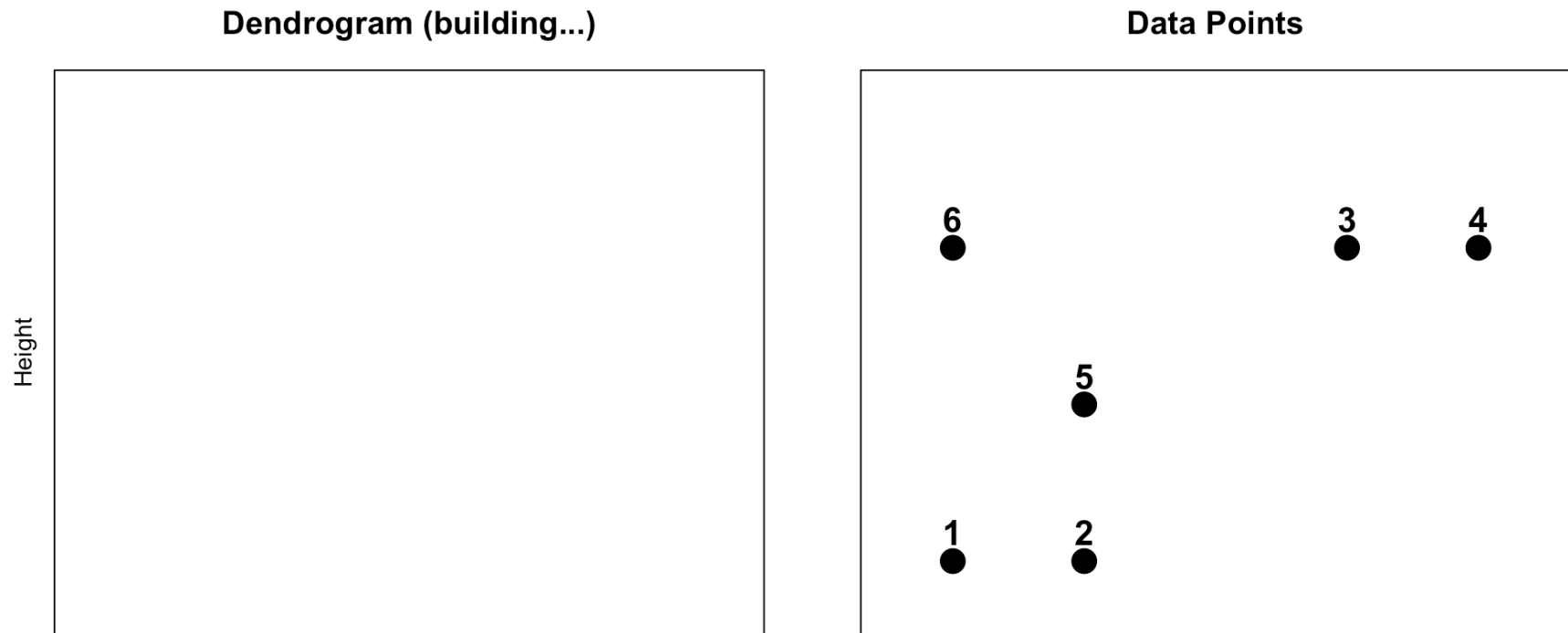
Point 6 joins {1,2,5} at height ≈ 0.36 (nearest neighbor is point 5) $\rightarrow$ 2 clusters: {1,2,5,6}, {3,4}

Single Linkage: Step 5 (final merge)



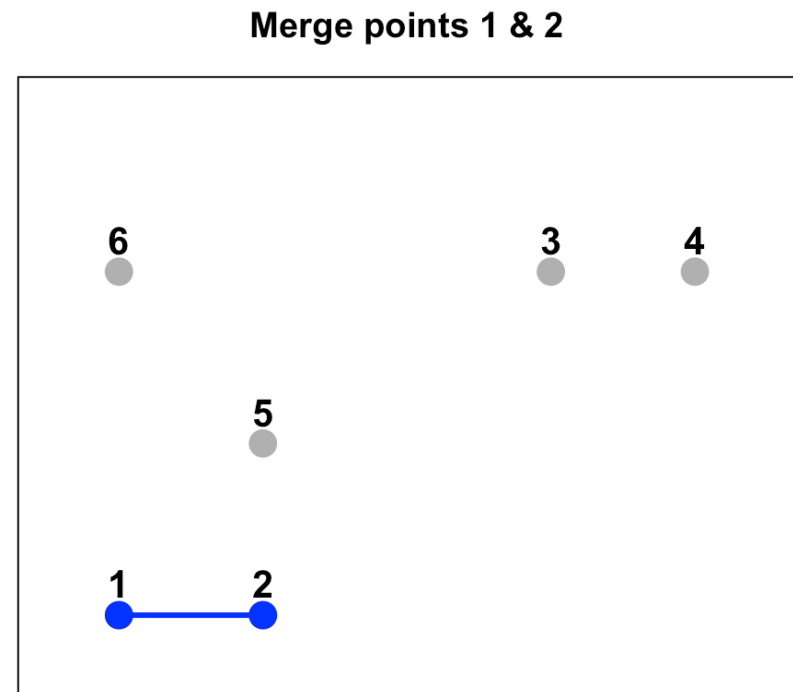
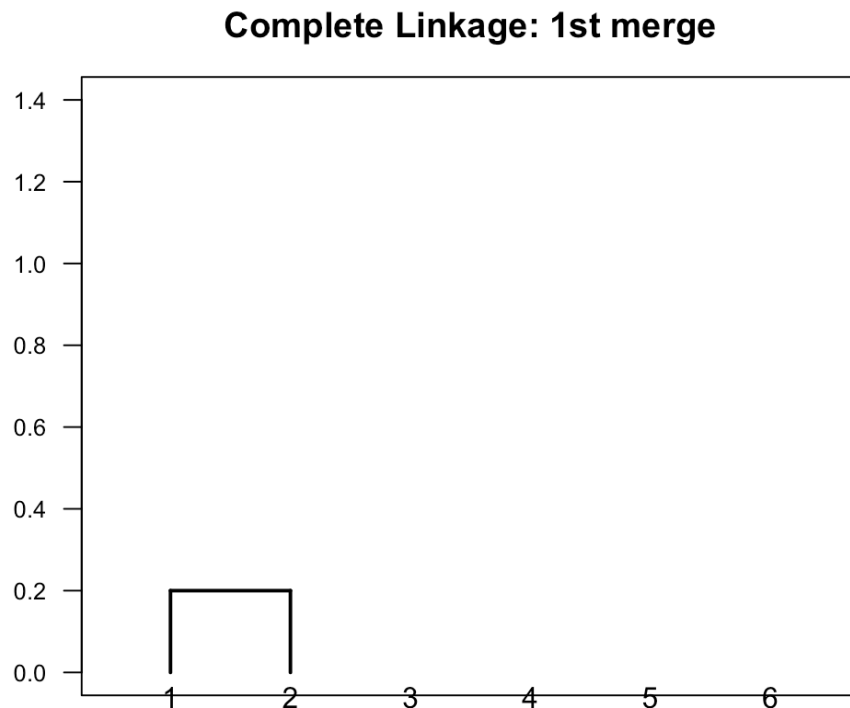
Final merge: Clusters $\{1,2,5,6\}$ and $\{3,4\}$ join at height $\approx 0.50 \rightarrow 1$ cluster

Complete Linkage: Initial Points



6 clusters (each point is its own cluster)

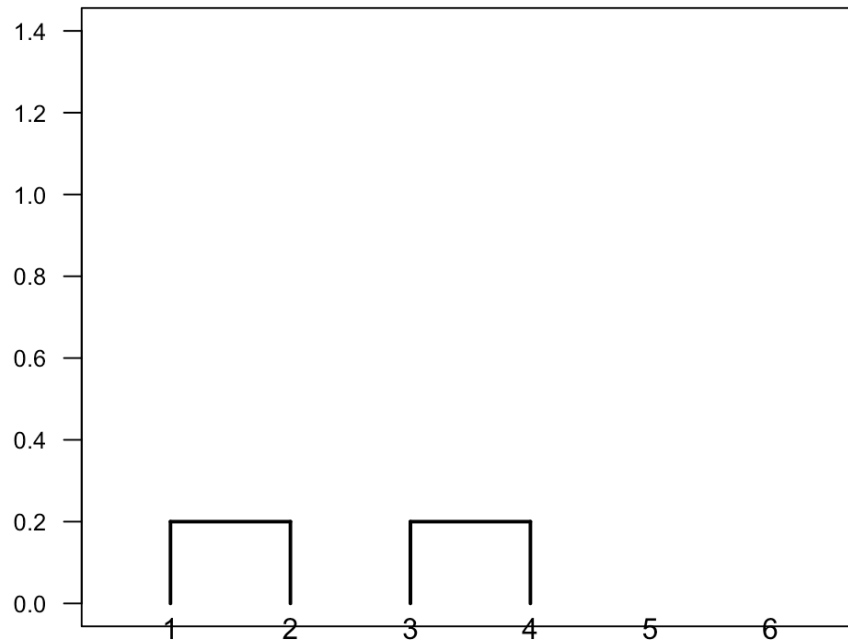
Complete Linkage: Step 1 (merge 1 & 2)



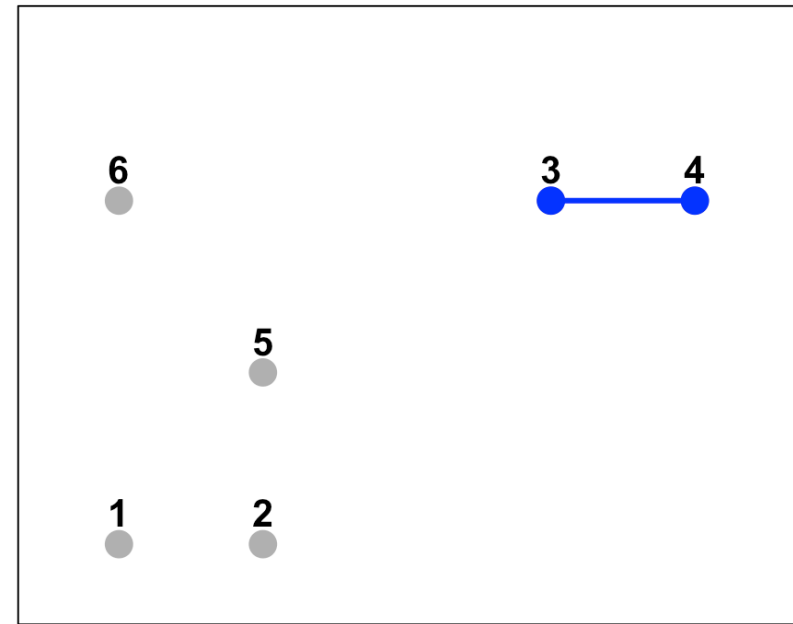
Closest pair: Points 1 & 2 (distance = 0.20) → 5 clusters

Complete Linkage: Step 2 (merge 3 & 4)

Complete Linkage: 2nd merge



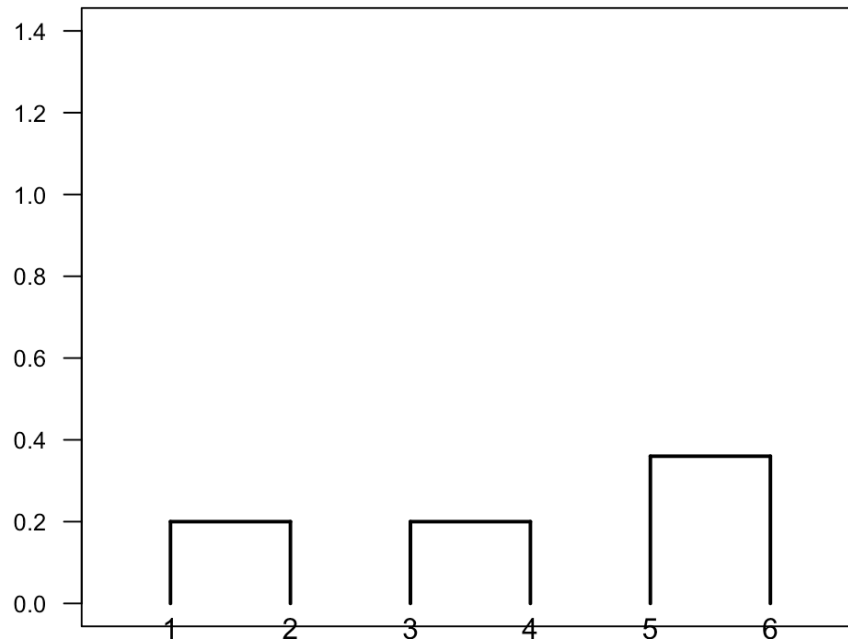
Merge points 3 & 4



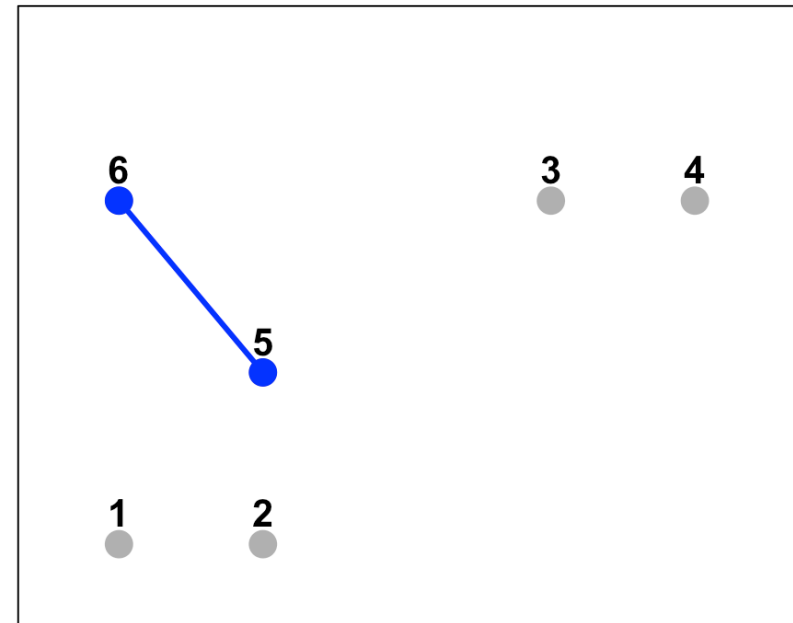
Next closest pair: Points 3 & 4 (distance = 0.20) → 4 clusters: {1,2}, {3,4}, {5}, {6}

Complete Linkage: Step 3 (merge 5 & 6)

Complete Linkage: 3rd merge



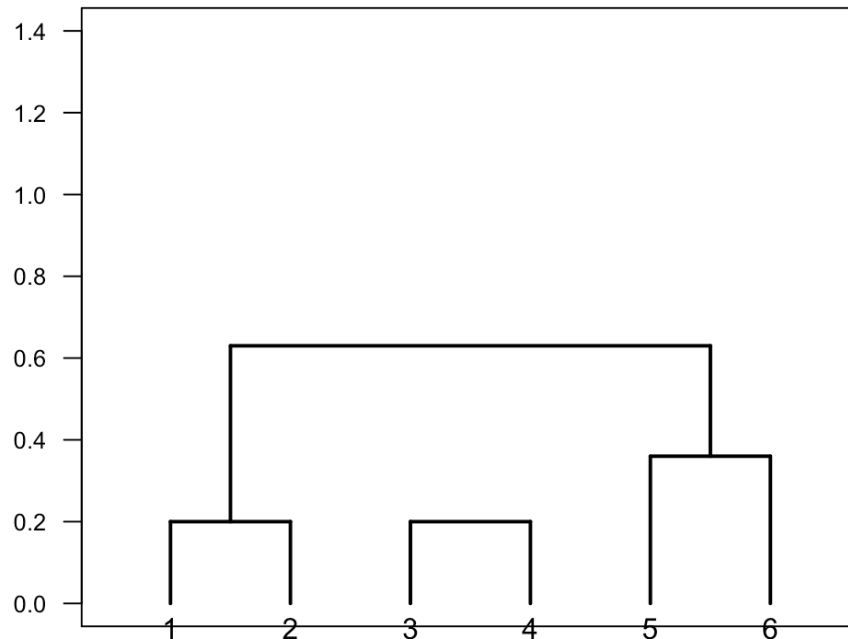
Merge points 5 & 6



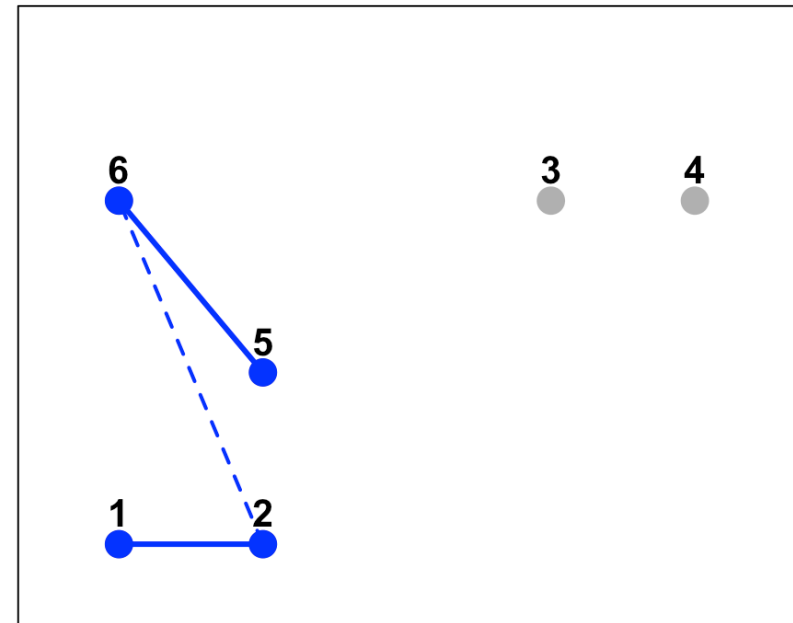
Points 5 & 6 form a cluster at height $\approx 0.36 \rightarrow$ 3 clusters: {1,2}, {3,4}, {5,6}

Complete Linkage: Step 4 ($\{1,2\}$ joins $\{5,6\}$)

Complete Linkage: 4th merge

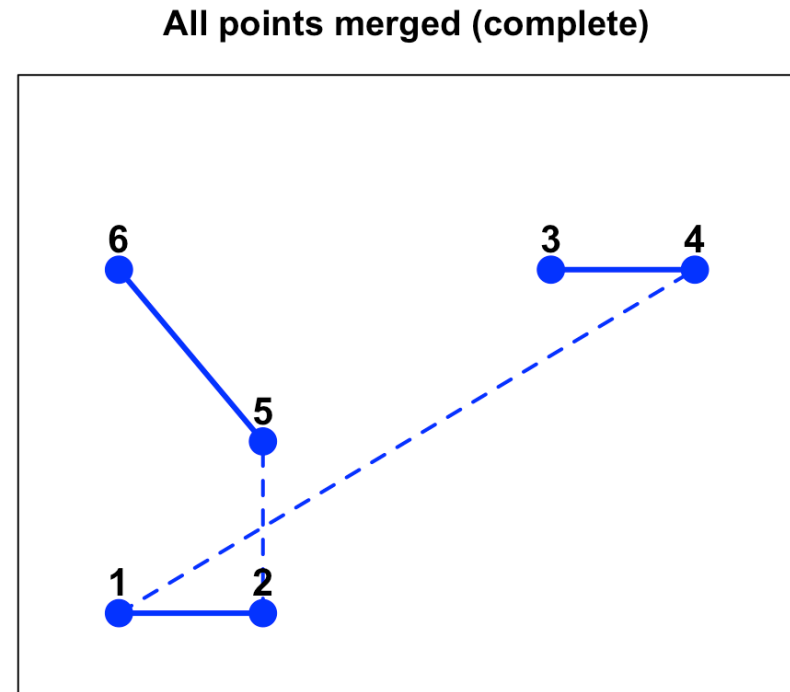
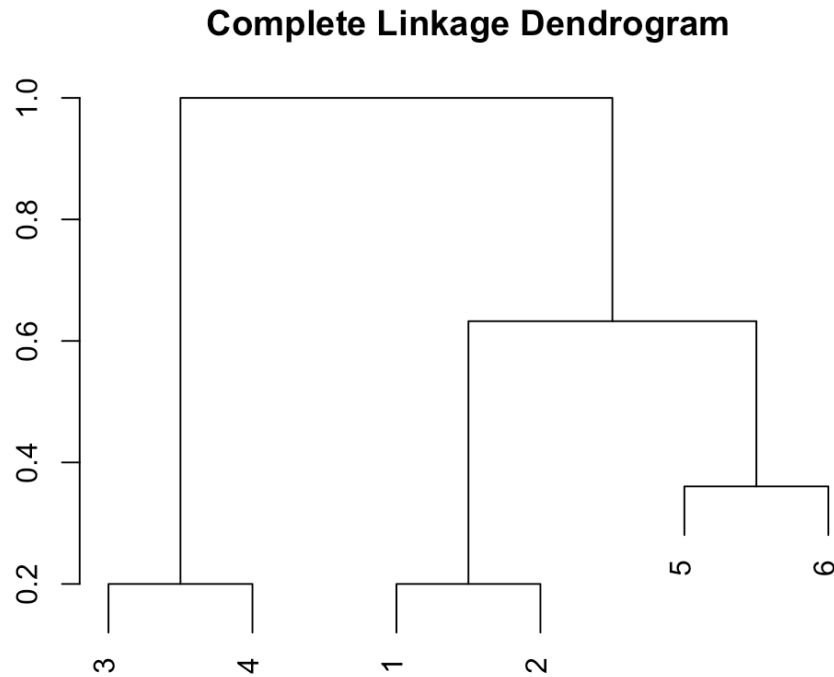


Clusters $\{1,2\}$ and $\{5,6\}$ merge



Clusters $\{1,2\}$ and $\{5,6\}$ merge at height equal to the maximum distance between any point in $\{1,2\}$ and any point in $\{5,6\}$, which in this case is the distance between points 6 and 2 (≈ 0.63) $\rightarrow$ **2 clusters:** $\{1,2,5,6\}$, $\{3,4\}$

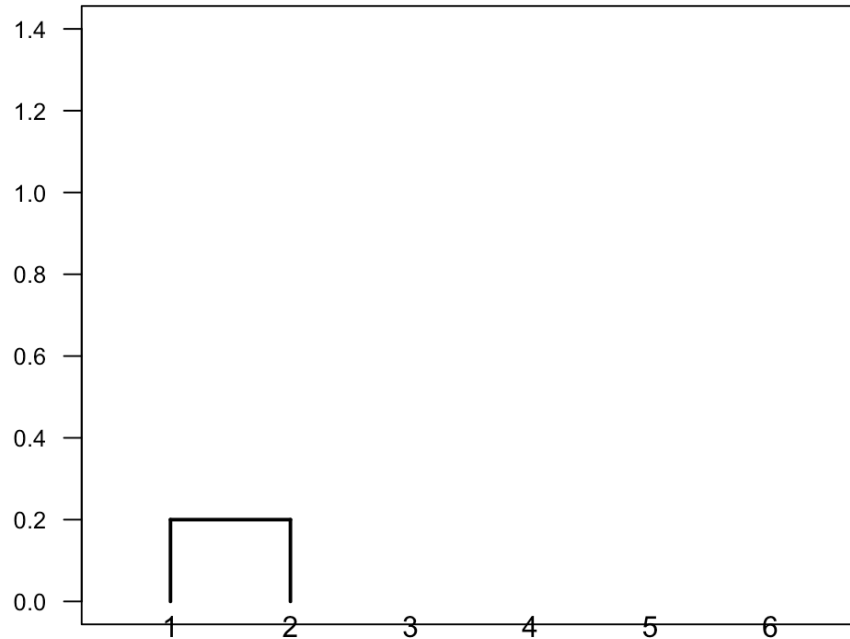
Complete Linkage: Step 5 (final merge)



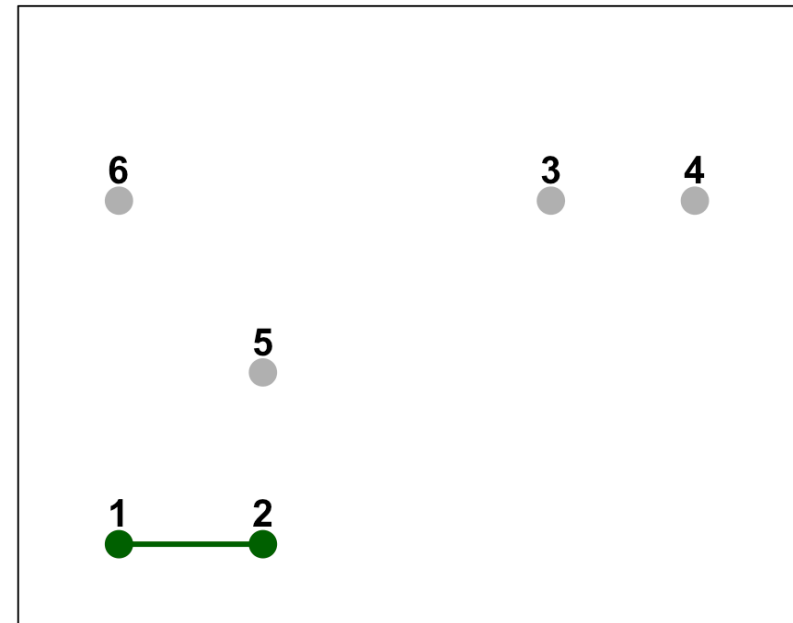
Final merge: Clusters $\{1,2,5,6\}$ and $\{3,4\}$ join at height $\approx 1.00 \rightarrow 1$ cluster

Average Linkage: Step 1 (merge 1 & 2)

Average Linkage: 1st merge

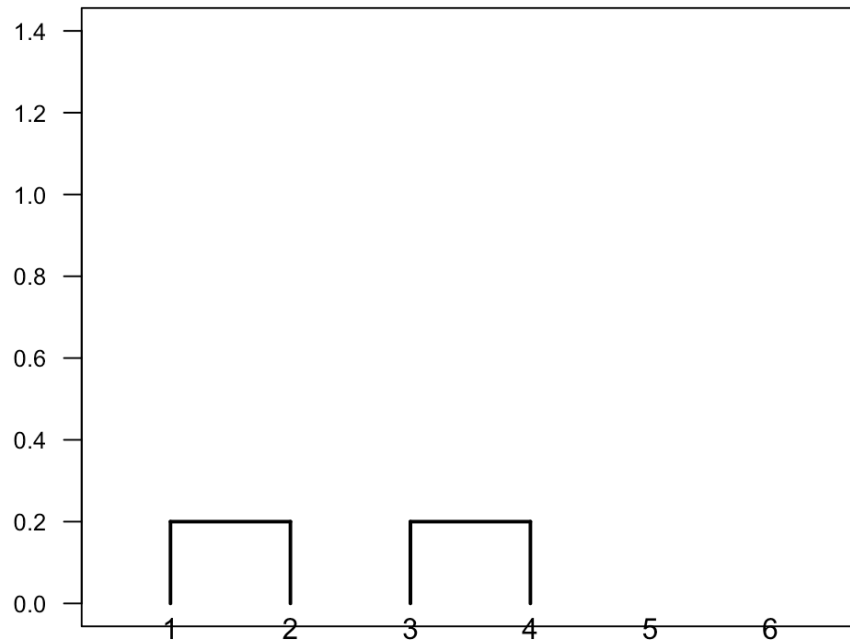


Merge points 1 & 2

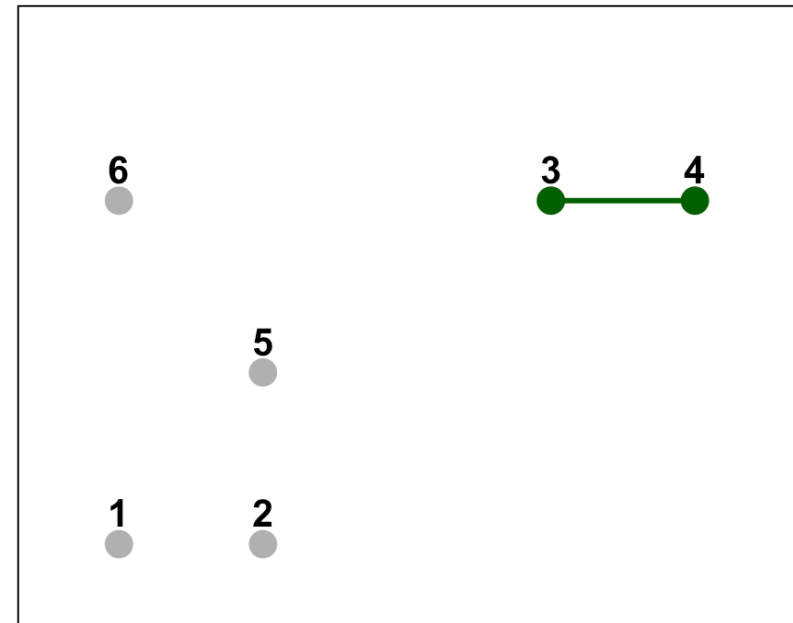


Average Linkage: Step 2 (merge 3 & 4)

Average Linkage: 2nd merge

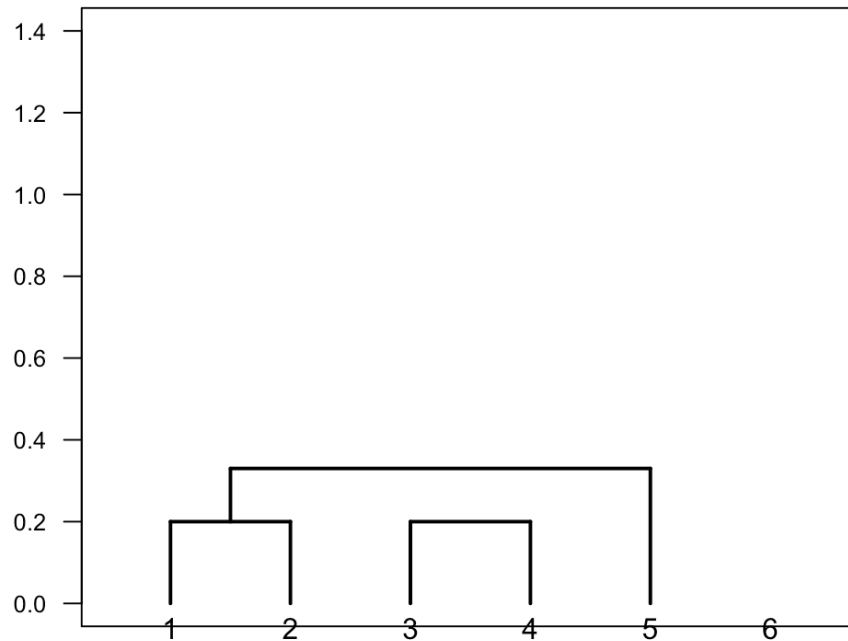


Merge points 3 & 4

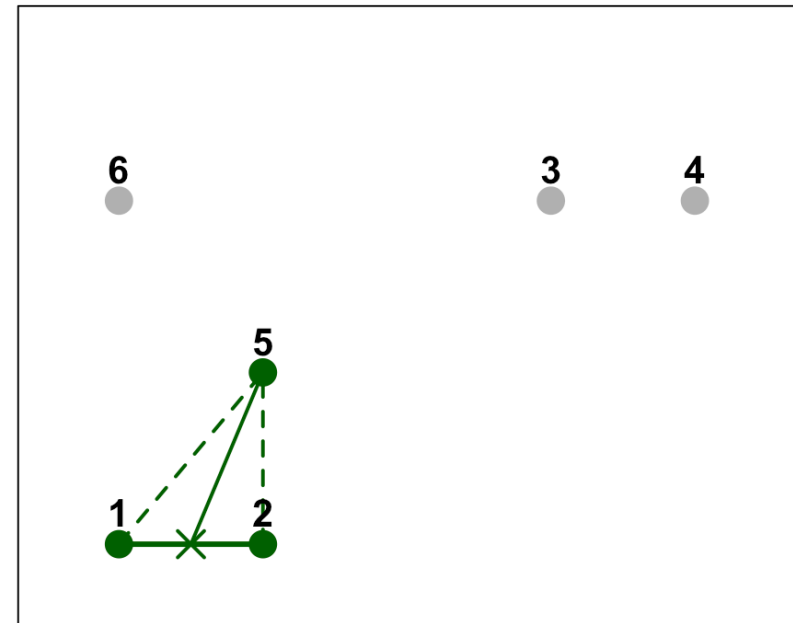


Average Linkage: Step 3 (5 joins {1,2})

Average Linkage: 3rd merge

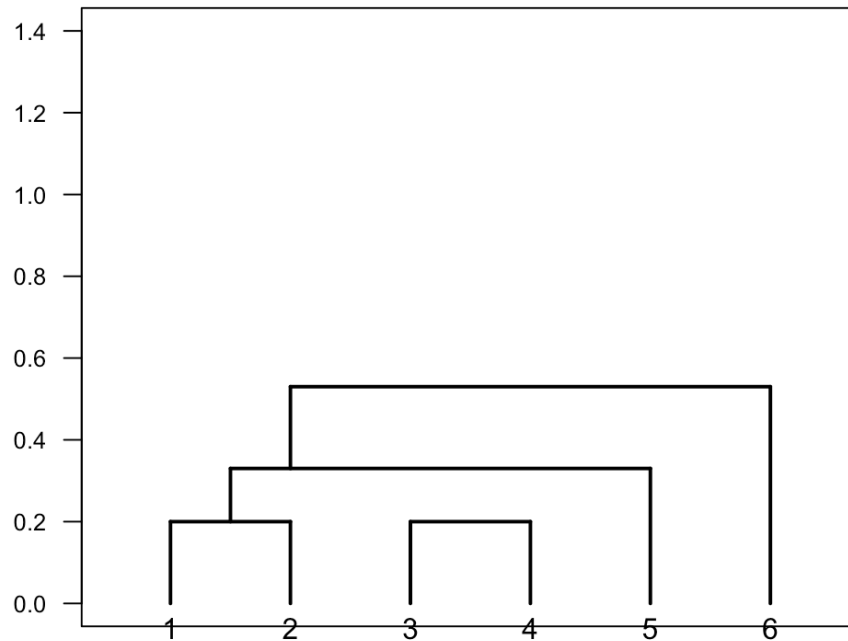


Point 5 joins cluster {1,2}

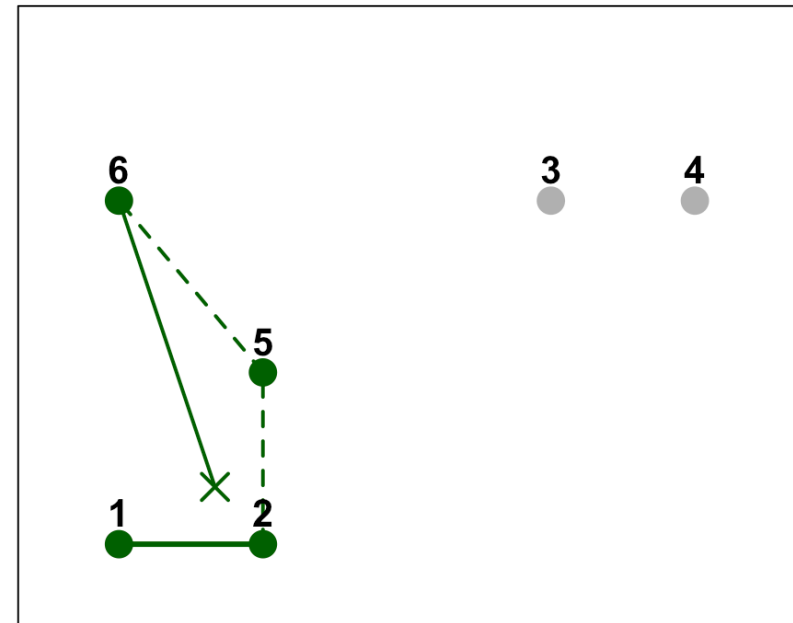


Average Linkage: Step 4 (6 joins {1,2,5})

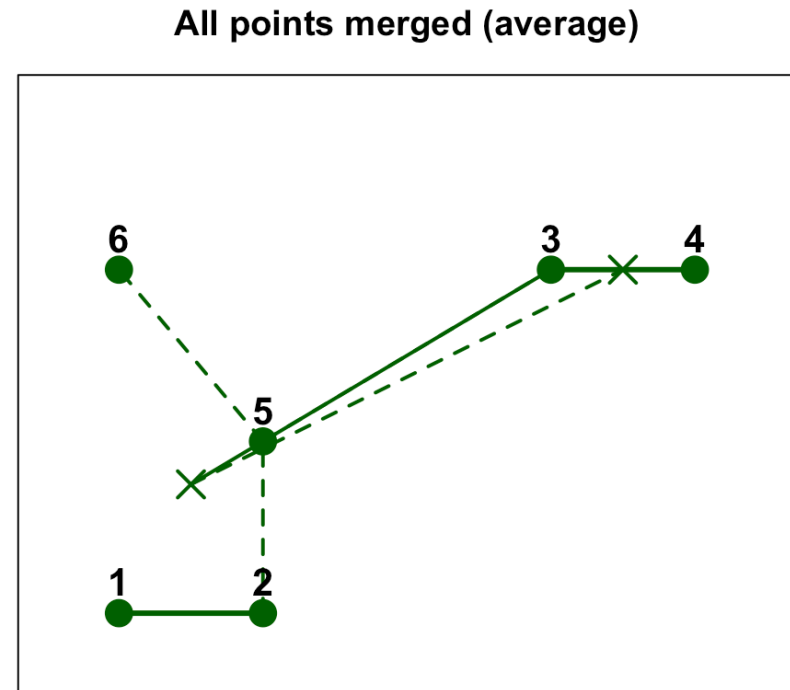
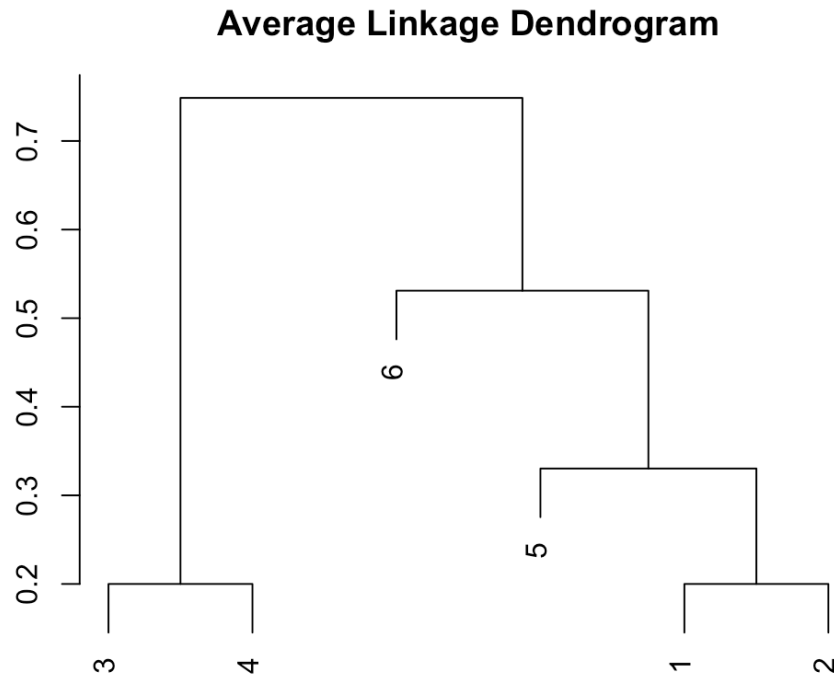
Average Linkage: 4th merge



Point 6 joins cluster {1,2,5}

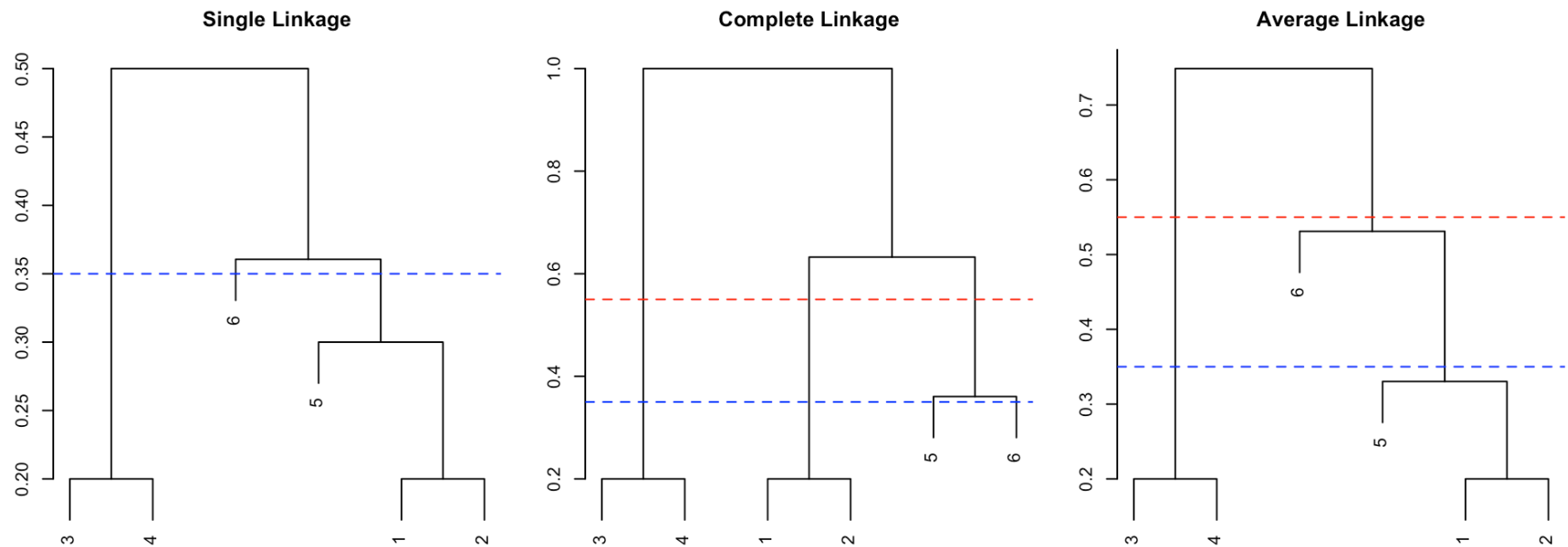


Average Linkage: Step 5 (final merge)



Final merge: Clusters $\{1, 2, 5, 6\}$ and $\{3, 4\}$ join at height $\approx 0.75 \rightarrow 1$ cluster

Comparison: Different Linkage, Different Clusters

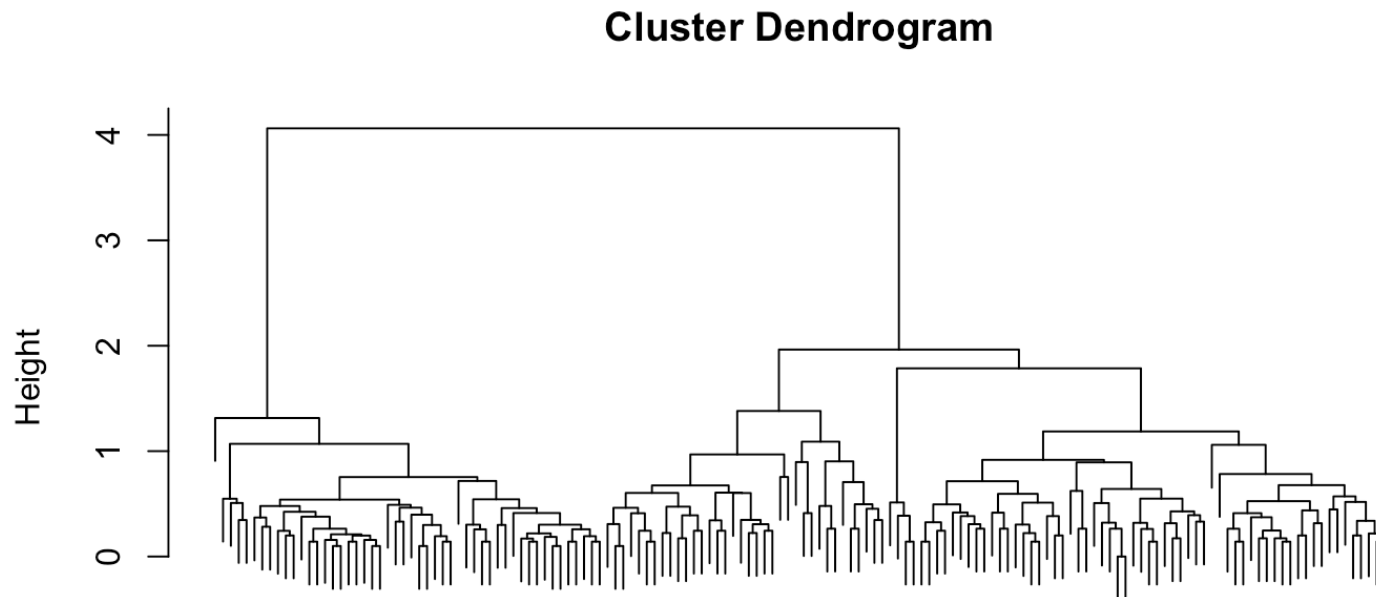


So the same data and same cut height give **different clusterings** for different linkage methods.

Hierarchical clustering in R

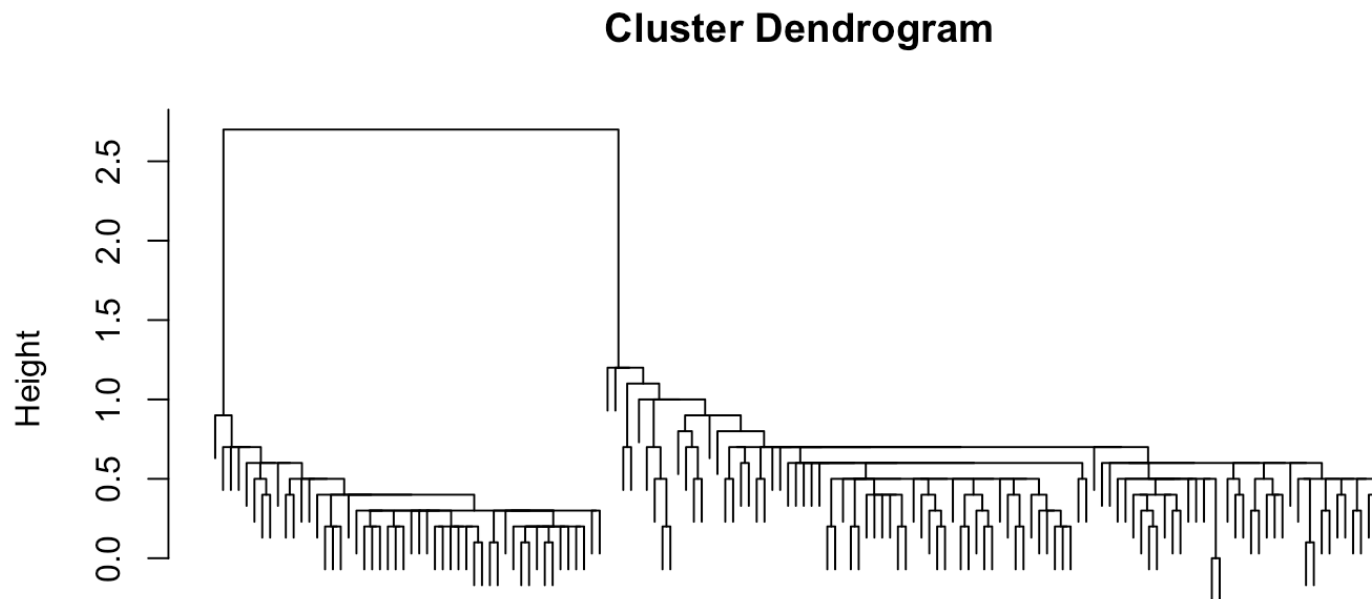
In Rstudio we can use the `hclust()`-function to perform hierarchical clustering.

```
dd <- dist(df_iris, method="euclidean")  
hc <- hclust(dd, method="average")  
plot(hc, labels = FALSE)
```



Example using Euclidean distance and single linkage

```
dd <- dist(df_iris, method="manhattan")  
hc <- hclust(dd, method="single")  
plot(hc, labels = FALSE)
```



Choosing the number of clusters

With a dendrogram, we cut it at some height to obtain K clusters
`cutree(hc, k = 3)` or `cutree(hc, h = 3.5)`.

How to choose the cut height or K ?

1. **Visual inspection:** Look for large jumps in height — cut just before a big jump.
2. **Domain knowledge:** If you expect K groups based on the problem context.
3. **Internal validation:** Use quality metrics (e.g., **silhouette** coefficient, **within-cluster sum of squares**) for different K .
4. **External validation (if labels available):** Compare at different heights to ground-truth using the **Adjusted Rand Index (ARI)**. Range $[-1, 1]$; 1 = perfect, 0 = random.

Compare hc and km clusters $K = 3$

Compare hc and km clusters K = 3

```
adjustedRandIndex(cutree(hc, k = 3), iris$Species)
```

```
## [1] 0.7591987
```

```
adjustedRandIndex(km$cluster, iris$Species)
```

```
## [1] 0.7302383
```

```
wcss(cutree(hc, k = 3))
```

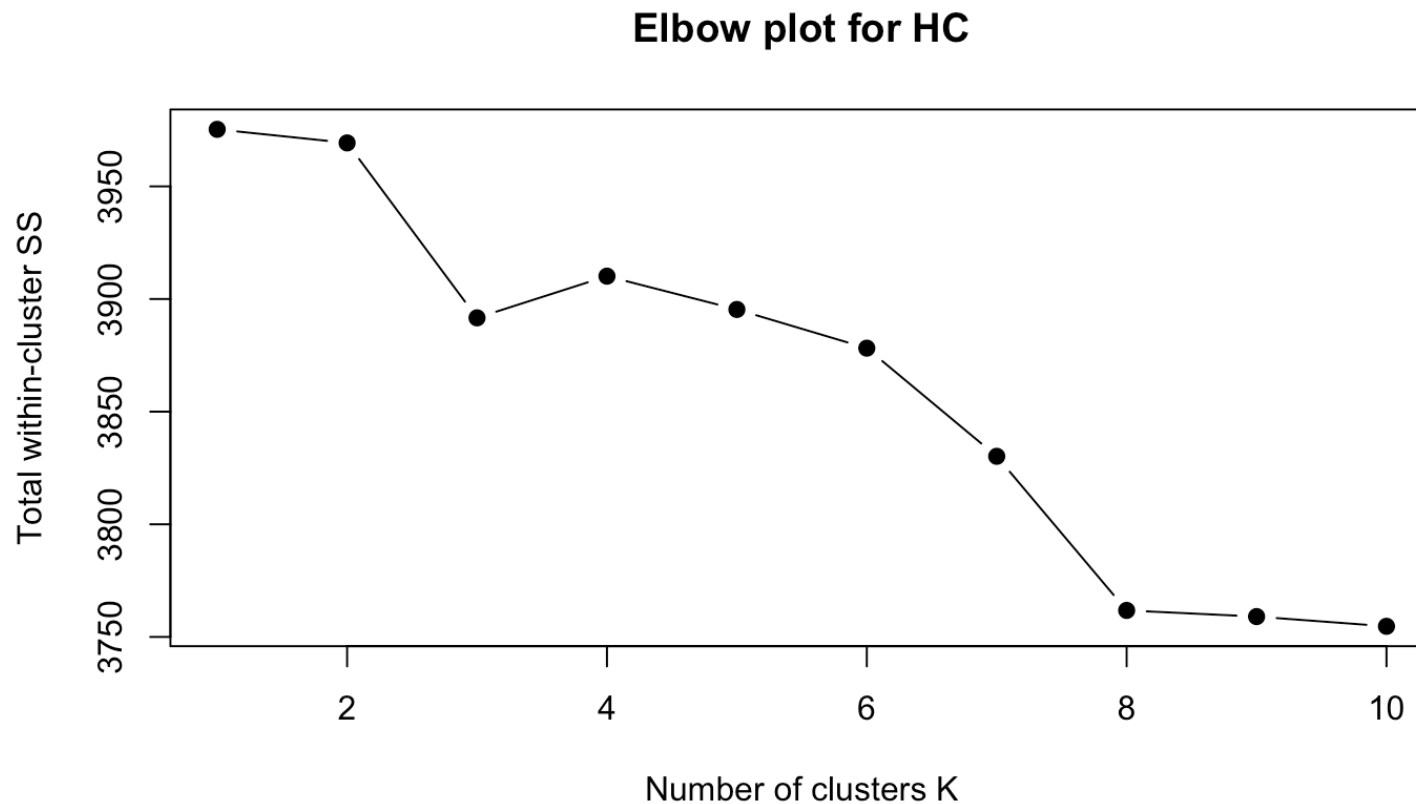
```
## [1] 3891.643
```

```
wcss(km$cluster)
```

```
## [1] 3924.729
```

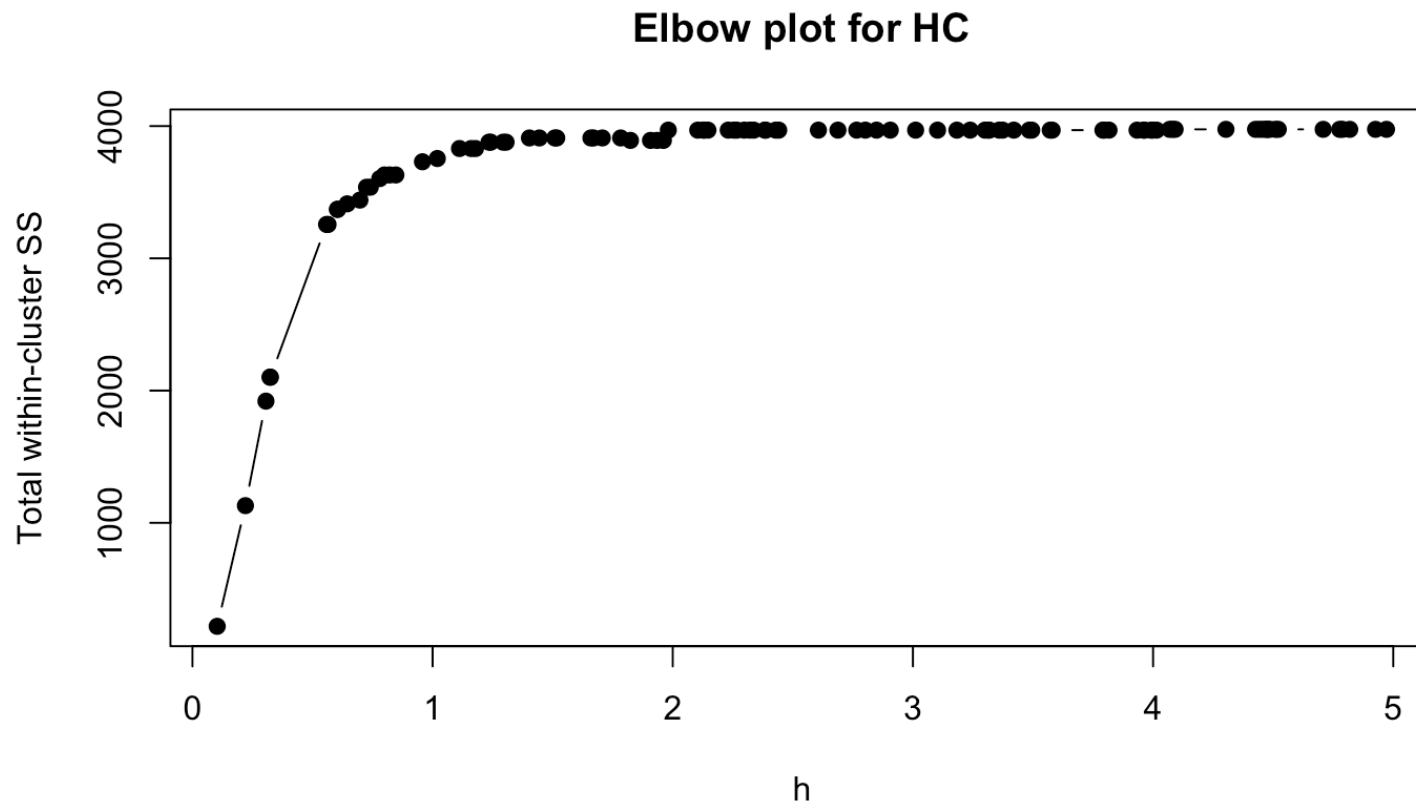
Elbow plots for WCSS and K

Do not need to recompute! Same dendrogram.



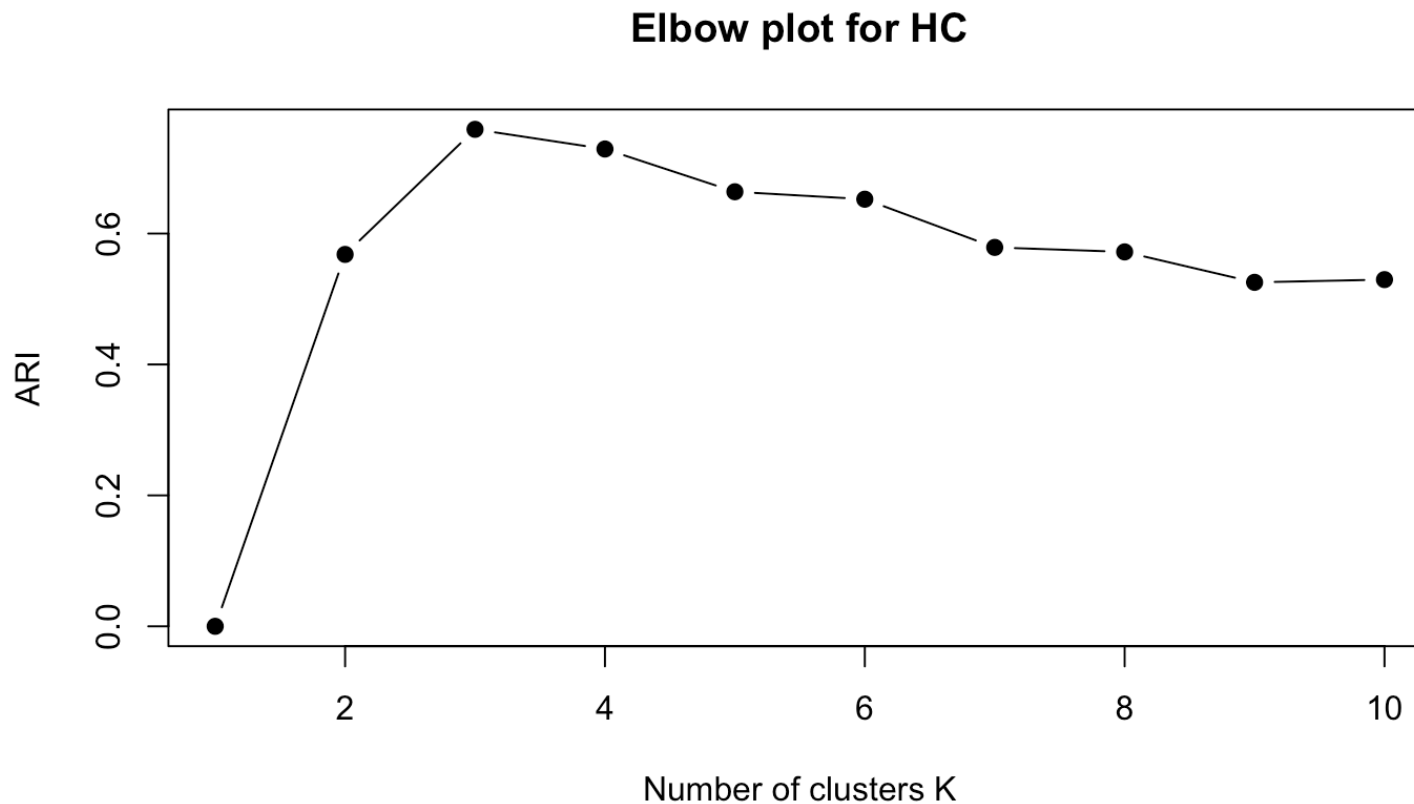
Elbow plots for WCSS and height

Do not need to recompute! Same dendrogram.



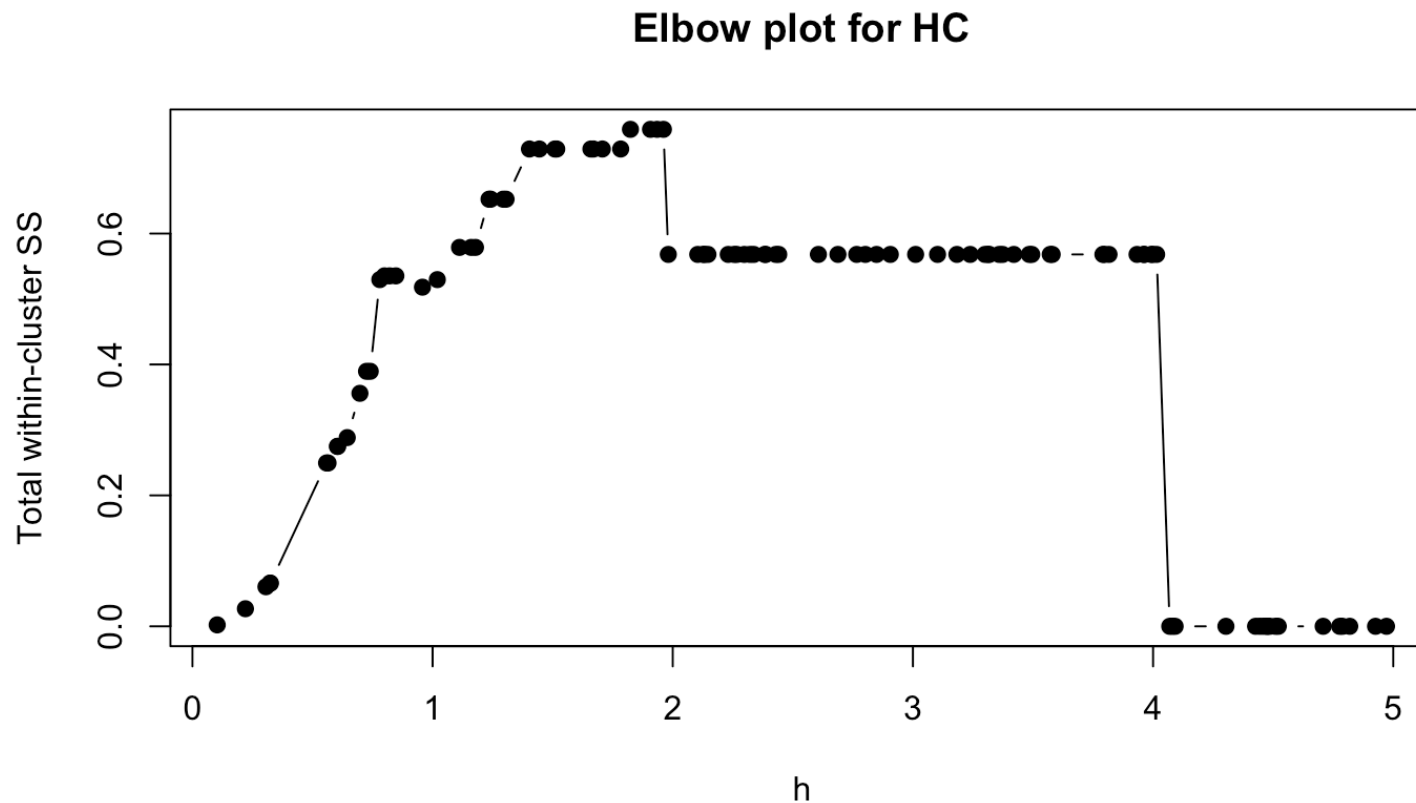
Elbow plots for ARI and K

Do not need to recompute! Same dendrogram.



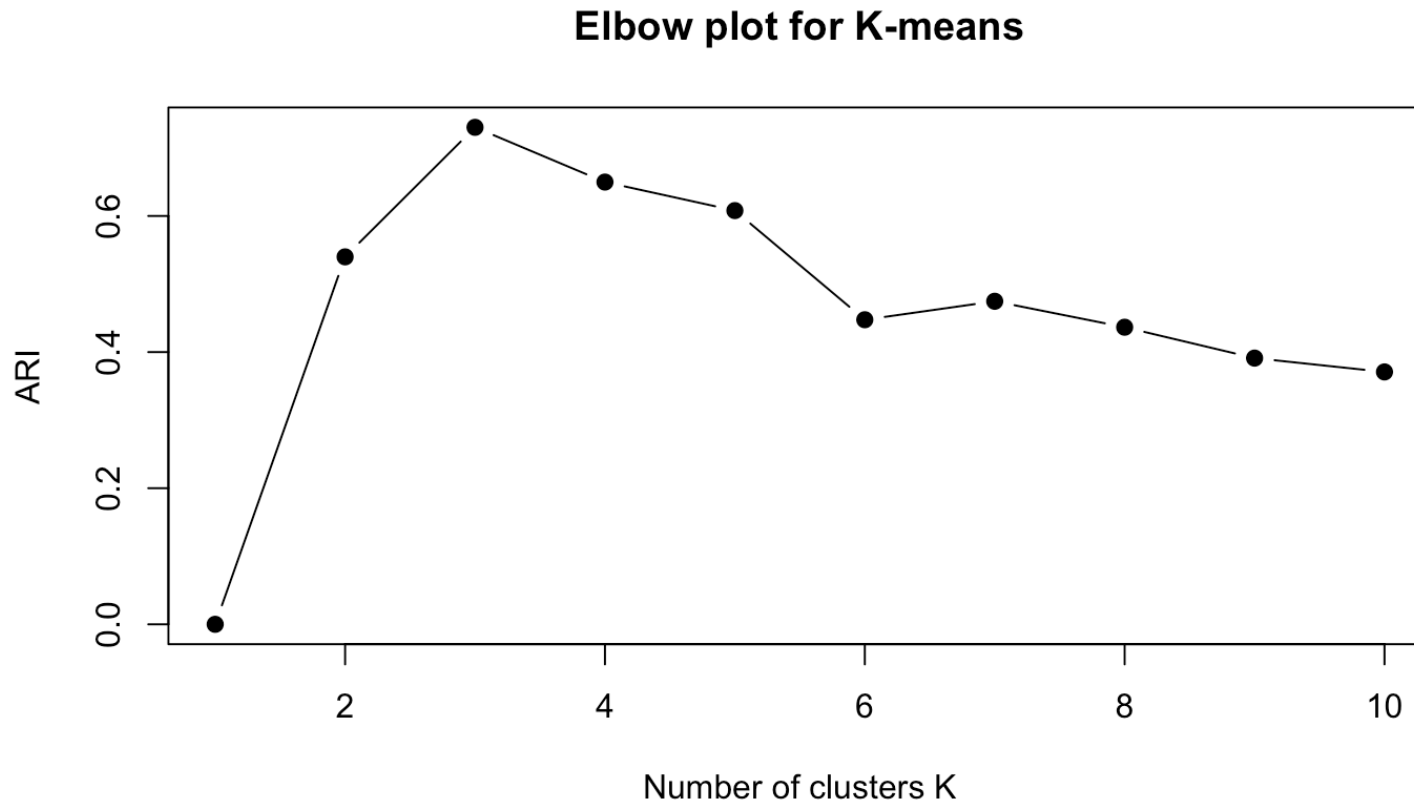
Elbow plots for ARI and height

Do not need to recompute! Same dendrogram.



Elbow plots for ARI and K for K-means

Do not need to recompute! Same dendrogram.



Silhouette coefficient

Internal validation (no ground-truth labels):

- **Silhouette coefficient** for observation i :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where $a(i)$ = average distance to points in same cluster,

$b(i)$ = average distance to points in nearest other cluster.

Range: $[-1, 1]$. Values near 1 = well-clustered, near 0 = borderline, negative = misclassified.

- **Average silhouette**: mean of $s(i)$ over all observations (higher is better).

Evaluating clustering quality with silhouette

```
sil <- silhouette(cutree(hc, k = 3), dist(iris[,1:4]))  
mean(sil[, 3]) # Average silhouette width
```

```
## [1] 0.5541609
```

```
sil <- silhouette(cutree(hc, h = 3), dist(iris[,1:4]))  
mean(sil[, 3]) # Average silhouette width
```

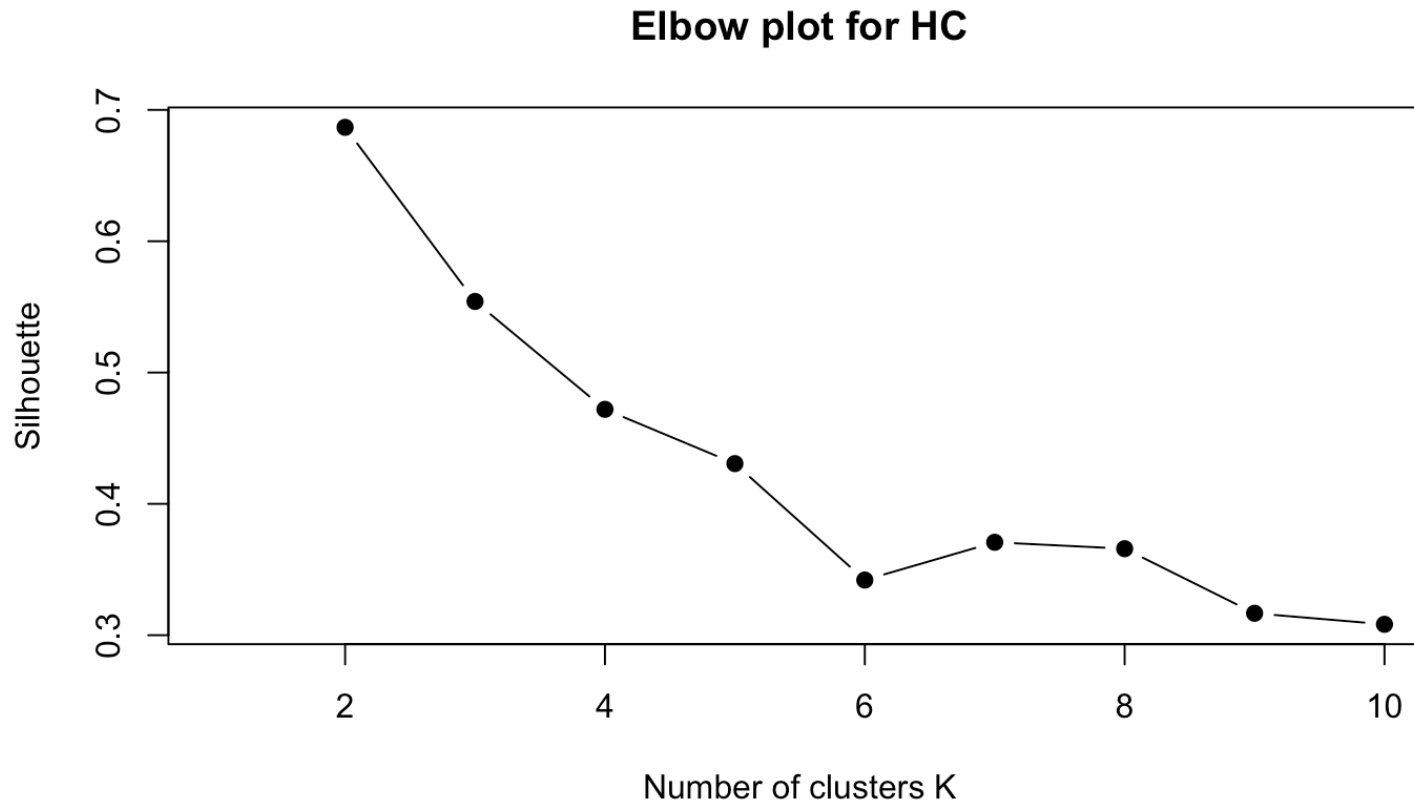
```
## [1] 0.6867351
```

```
sil <- silhouette(km$cluster, dist(iris[,1:4]))  
mean(sil[, 3]) # Average silhouette width
```

```
## [1] 0.3223781
```

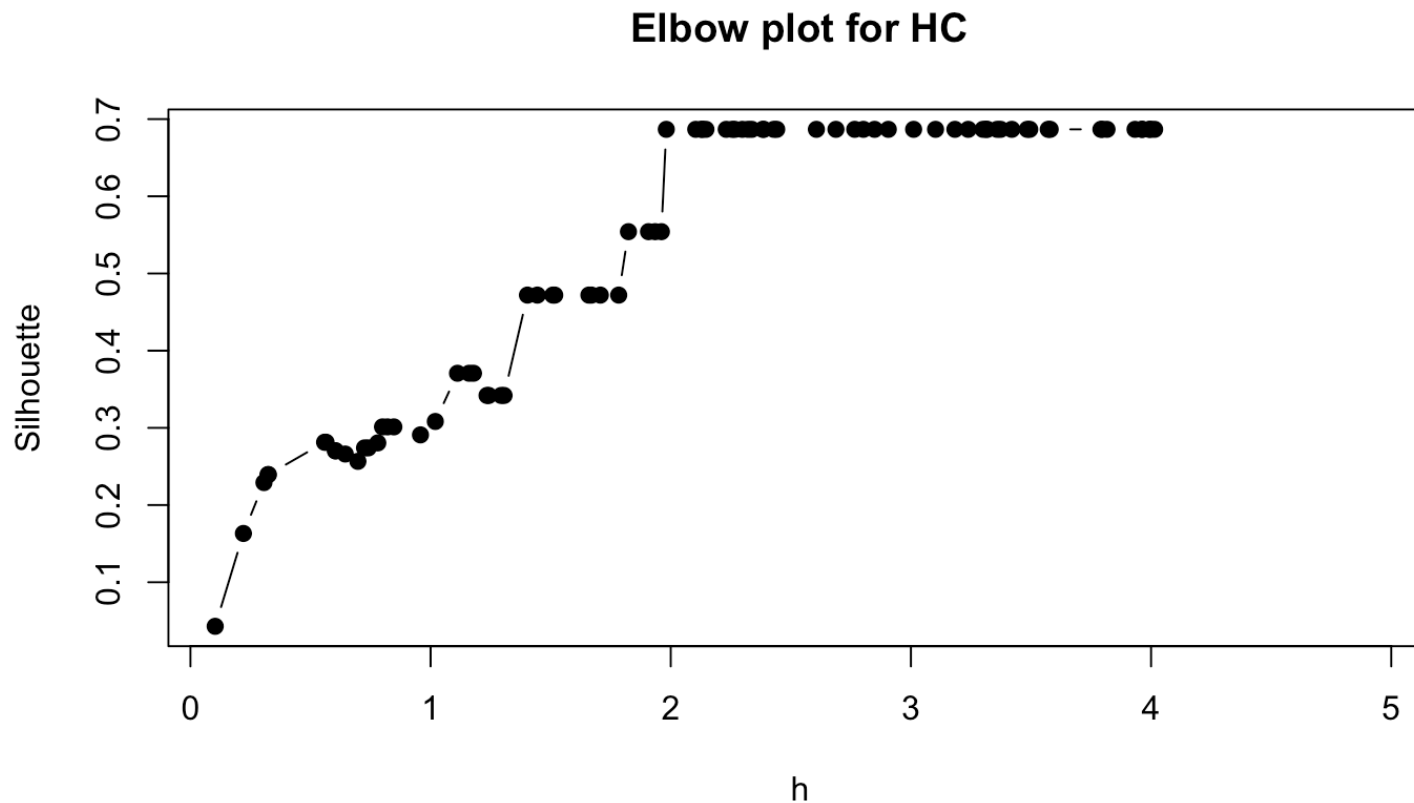
Elbow plots for silhouette and K

Do not need to recompute! Same dendrogram.



Elbow plots for silhouette and height

Do not need to recompute! Same dendrogram.



Other clustering methods not covered

Model-based clustering:

- **Mixture models:** Fit mixtures of distributions to data using the EM algorithm; provides soft cluster assignments (probabilities) and statistically principled model selection (BIC).
- R: `Mclust` (package `mclust`).

Other clustering methods not covered

Density-based clustering:

- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise): Finds clusters of arbitrary shape based on local density; automatically detects noise/outliers; does not require specifying K in advance.
- R: `dbscan()` (package `dbscan`).

Other clustering methods not covered

Graph-based clustering:

- **Spectral clustering:** Uses eigenvectors of the graph Laplacian to embed data in low-dimensional space, then applies k-means; effective for non-convex clusters.

Other clustering methods not covered

Soft clustering:

- **Fuzzy c-means:** Assigns each observation a membership degree (between 0 and 1) to each cluster instead of hard assignment; useful when boundaries are uncertain.
- R: `cmeans()`

Others: Self-organizing maps (Kohonen maps), dynamic time warping for time-series clustering.